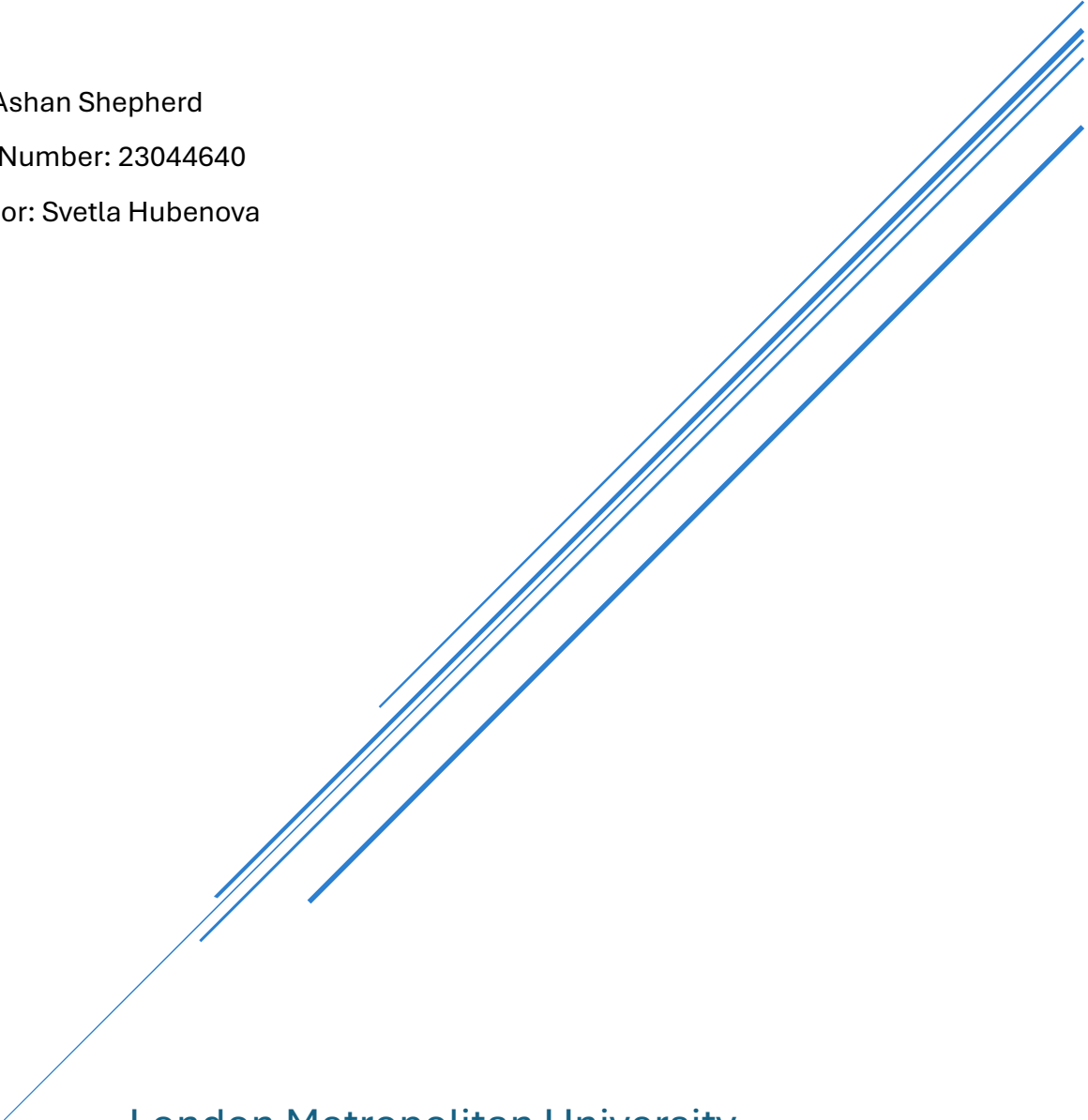


THE CLASSIFICATION OF EXOPLANETS USING MACHINE LEARNING TECHNIQUES

Author: Ashan Shepherd

Student Number: 23044640

Supervisor: Svetla Hubenova



London Metropolitan University
MCs Data Analytics

Contents

Abstract	4
Chapter 1 - Introduction	5
1.1 Introduction	5
1.2 Aims and Objectives	7
Chapter 2 - Previous Literature	10
2.1 Discovery Methods	10
2.2 Exoplanet Classification	12
2.3 CRISP-DM Methodology	12
2.4 Machine Learning	13
2.4.1 Supervised Machine Learning	14
2.4.2 Unsupervised Machine Learning	15
2.4.3 Reinforcement Machine Learning	16
2.5 Machine Learning Algorithms	17
2.5.1 K-Means Clustering	17
2.5.2 Support Vector Machine (SVM)	18
2.5.3 Decision Tree	20
2.5.4 K-Nearest Neighbour (KNN)	21
2.6 Model Comparison	22
Chapter 3 - Methodology	26
3.1 Business Understanding	26
3.1.1 Software	27
3.2 Data Understanding	28
3.3 Data Preparation	28
3.4 Modelling	31
3.5 Evaluation	32
3.6 Deployment	32
Chapter 4 - Analysis	33
4.1 Data Understanding	33

4.2 Data Preparation	42
4.2.1 Handling of Missing Data	42
4.2.2 Data Transformation	45
4.2.3 Response Variable	47
4.3 Modelling	49
4.3.1 K-Means	49
4.3.2 Support Vector Machine (SVM)	50
4.3.3 Decision Tree	51
4.3.4 K-Nearest Neighbour (KNN)	51
4.4 Model Comparison	54
Chapter 5 - Conclusion	57
5.1 Conclusion	57
5.2 Recommendation for Future Work	58
References	59
Appendix	65

Figures

Figure 1 - Exoplanet discovery method rate per year with discovery method (NASA Exoplanet Archive, n.d.)	10
Figure 2 - K-means Cluster Formula for the K-Means Clustering Algorithm	17
Figure 3 - Hyperplane Formula for the SVM Algorithm	18
Figure 4 - D-dimensional Real Space Formula for the SVM Algorithm	18
Figure 5 - Kernel Function Formula for the SVM Algorithm	19
Figure 6 - Entropy Formula for Decision Tree Algorithm	20
Figure 7 - Gini Index Formula for Decision Tree Algorithms	21
Figure 8 - The Euclidean Distance Formula for the KNN Algorithm	21
Figure 9 - Accuracy Formula for Model Comparison	24
Figure 10 - F1 Score Formula for Model Comparison	24
Figure 11 - Precision and Recall Formula for Model Comparison	24

Figure 12 - Equation for Min-max Scaling	30
Figure 13 - Formula for Standardisation	30
Figure 14 - Histogram of the distribution of data for each selected variable	38
Figure 15 -Box Plot of the distribution of data for each selected variable	41
Figure 16 - Correlation Matrix Between Variables	43
Figure 17 - Scatter Plots of Variables with the Highest Correlation	44
Figure 18 - Formula for Density	45
Figure 19 - Histograms of the Dataset Variables after Log-Transformation	46
Figure 20 - The Distribution of Planet Types Before SMOTE was Applied	48
Figure 21 - 3d Clustering Results Showing Four Groups	49
Figure 22 - Confusion Matrix for SVM	50
Figure 23 - Decision Tree	51
Figure 24 - Confusion Matrix for the Decision Tree Model	52
Figure 25 - The Elbow Method for Finding the Optimum Value of K	53
Figure 26 - Confusion Matrix for the KNN Model	54
Figure 27 - ROC Curve Visualisation of the SVM, KNN and Decision Tree Models	56

Tables

Table 1 - Variables selected for Machine Learning Classification	33
Table 2 - Percentage of Missing Values in the Dataset	42
Table 3 - Ranges of Measurements for the Creation of the Target Variable Planet Type	47
Table 4 - The Classification Report of the KNN, Decision Tree and SVM Model for Model Comparison	55

Abstract

This project aims to successfully use machine learning to classify exoplanets correctly into groups based on key variables from the NASA Exoplanet Archive dataset. Classification and clustering machine learning models, including the k-means, decision tree, SVM and KNN, were used. The study will determine the most reliable classification process for categorising exoplanets into NASA's four main types: Gas Giant, Neptunian, Super-Earth, and Terrestrial. It was ensured that missing values were identified and correctly dealt with through different imputation techniques or by removing them entirely. Among the three supervised models used throughout this project, SVM performed best in accurately classifying exoplanets, identifying all four types.

1. Introduction

1.1 - Introduction

Space exploration has changed from scientific curiosity to an important area for geopolitical strategy and economic development. Satellites are vital in modern society as they play key roles in communication, navigation, weather forecasting, and military operations (Aslam & Afridi, 2024). Astronomy has also influenced religion, guided explorers and created philosophical questions about our existence. A natural curiosity of our existence questions if there is another planet in our universe which can support and maybe already has existing life, a question which has been thought and researched for hundreds of years (Clayton, Brychan and Sohail, 2019).

The strive to understand our place in the universe has led and advanced humans from observing constellations with the naked eye to sending complex space telescopes into space (Fan, 2024). The first planet to be discovered in our solar system was the planet Uranus in 1781 with a telescope by William Herschel, and by 1846, the last of the eight planets was discovered. Outside our solar system exist other planets referred to as exoplanets, in which the first ones were discovered in 1992 by astronomers Aleksander Wolszczan and Dale Frail. This was said to be the start of planet detection becoming a major research area in astrophysics (Wolszczan & Frail 1992). The discovery of exoplanets that orbit stars beyond our solar system represents a huge leap in the pursuit of habitable planets and life (Impey, 2023). These first exoplanets were discovered using the pulsar timing method, which measured the variations in the timing of pulses caused by the gravitational influence of an orbiting planet (Kramer, 2017). The first exoplanet discovered orbiting a star like our sun was in 1995 using the radial velocity method in which astronomers detect planets indirectly by measuring the wobbling of a star due to the gravitational tug of orbiting planets (Wright et al., 2012). Since the first discovery, there have now been 5,834 confirmed exoplanets through various techniques at the time of this study. These different methods of

exoplanet discovery provide different information about the planets, such as their radius, mass and orbital period. This creates issues when analysing them, as planets discovered by one method may contain information that another detection method does not provide. Therefore, statistical analysis can become a challenge as there are many missing values present in the data. For example, planets detected by the transit technique can measure radius, whilst the radial velocity technique measures minimum mass (Lalande, Tasker and Doya, 2024). Detecting planets with multiple discovery techniques would be ideal for creating a full archive database, but it is often not possible.

Planets being detected can be classified into four major categories according to NASA. Gas giant, Neptunian, super-Earth, and terrestrial are planet types which exoplanets and planets in our solar system can be described as. Size, temperature, and surface types are features which influence what category a planet is put into. Gas giants are the size or larger than the size of the planets Saturn and Jupiter in our solar system. They are in close orbits with their stars, making them significantly hot. Neptunian planets are similar in size to Neptune and Uranus in our Solar System, with atmospheres dominated by hydrogen or helium. Super-Earth exoplanets are typically rocky planets which may not have an atmosphere. They have a greater mass than Earth but less mass than Neptune. Terrestrial exoplanets are earth-sized or smaller and are made of rock, silicate, water or carbon (NASA, n.d.).

The number of exoplanet discoveries has increased at a rapid rate, and the amount of complex data has also increased. This has created a shift in the requirement for more advanced analytical methods to be used (Airlangga, 2024). The launch of satellites like Kepler has created a new era of exoplanet detection, one in which automated scientific observations and vast amounts of data are present. This has increased the need for machine learning to efficiently process and analyse exoplanets, helping scientists uncover patterns and make accurate predictions (Clayton, Brychan, and Sohail, 2019). Exoplanet identification has been a time-intensive task engaged by experts in the field with usually expensive and specialised equipment (Sturrock, Sturrock and Rafiqi, 2019). With the count

of exoplanet discoveries rising rapidly, alternative techniques are required to keep up with the amount of data captured and archived. It must also be noted that the validation process of the candidates discovered in the Kepler mission is still ongoing. Ruling out false positives is a time-consuming task but necessary as discovery techniques often falsely detect new planets (Malik, Moster and Obermeier, 2021). This is another task which machine learning can and has tackled (Jin, Yang and Chiang, 2022).

The process of building strong models to give accurate results in machine learning can be structured on the CRISP-DM methodology which this study will follow. Different computer software and programming languages will be used to execute the aims and objectives of the study.

1.2 - Aims and Objectives

This project aims to successfully use machine learning to classify exoplanets correctly into groups based on key variables from the NASA Exoplanet Archive dataset. Therefore, various classification models will be implemented to train and test the data, comparing their accuracy. If successful, the study will determine the most reliable classification process for categorising exoplanets into NASA's four main types: Gas Giant, Neptunian, Super-Earth, and Terrestrial.

To achieve the aim of creating a successful machine-learning algorithm for the classification of exoplanets, the objectives were created using the SMART approach (Specific, Measurable, Achievable, Relevant, and Time-based). Based on the aim, the objectives are to:

- Conduct a comprehensive review of literature on exoplanet classification and machine learning applications within two months to ensure a strong theoretical foundation for the study, allowing the identification of research gaps.

- Extract, transform and load the dataset, handling missing values, normalising data, and selecting relevant features within the first two months making sure there is a high-quality input for the models.
- Implement and evaluate at least three classification algorithms for classification using performance metrics, completed in the third month of the project.
- Optimise the best-performing models by fine-tuning the necessary hyperparameters within the month after the initial analysis.
- Compare and justify the final model selection based on the observed results, and document findings within the final month of the project,

Creating objectives using the SMART method ensures goals are clearly defined, realistically attainable based on resources, relevant to the aim and that a timeframe is set for when each goal is to be achieved (Lawlor and Hornyak, 2012).

One of the objectives is to identify ways to clean and prepare the NASA Exoplanet Archive dataset for analysis. Due to the presence of many null values in the data, various approaches can be employed to address these issues and optimise the dataset for analysis. Null values can be removed, which will significantly decrease the number of rows available for machine learning, or algorithms can be developed to estimate the missing properties (Lalande, Tasker, and Doya, 2024). This objective also aims to measure which explanatory variables are the strongest predictors of planet classification. Additionally, data will be standardised to ensure fair comparisons between variable values during the implementation stage of the project.

Another objective is to compare different machine-learning algorithms implemented with the dataset. Supervised and unsupervised machine-learning techniques will be tested against the data, and their effectiveness will be compared. Their performances will be evaluated based on metrics such as accuracy, precision, recall, and confusion matrices.

Moreover, results from the classification algorithms will be compared with NASA's exoplanet classification structure and other previous research

Academic objectives from this project include using several analytical skills and techniques acquired during the course. This includes implementing certain methods taught, ranging from machine learning algorithms, statistics and soft skills.

2 - Literature Review

2.1 - Discovery Methods

To gather planetary data, various techniques have been used to discover new exoplanets. The NASA Exoplanet Archive dataset contains confirmed planets discovered by 11 different methods. The transit method has produced the most exoplanet discoveries to date, with the radial velocity method providing the second largest (NASA Exoplanet Science Institution). These methods, along with the pulsar timing, gravitational microlensing, direct imaging and astrometry methods, have been widely used in the ongoing ground and space-based exoplanet search projects (Jia and Gang, 2012).

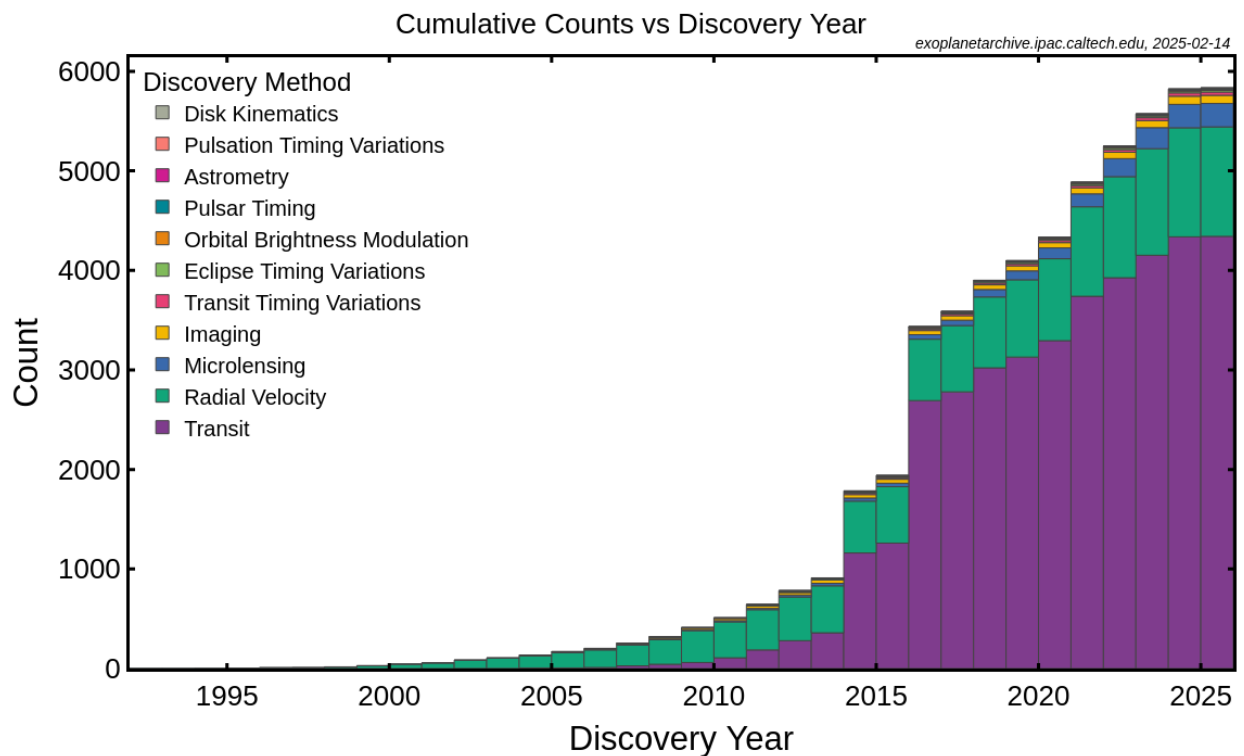


Figure 1: Exoplanet discovery method rate per year with discovery method (NASA Exoplanet Archive, n.d.)

The transit method has discovered more than 3,000 exoplanets. This method works when the observer, the planet, and the host star are close to collinear. When the planet is between

the observer and the host star, the planet will pass through the front of the host star, causing part of the star's light to be blocked by the planet. This blocking will be shown in the photometric measurement of the star, called a transit event (Charbonneau, Brown and Latham, 2000). Variables which the transit method can directly measure are planet radius, orbital period, transit depth and transit duration. (Deeg and Alonso, 2018). Radial velocity, which works by measuring the tiny wobbles in a star's motion caused by the gravitational pull of an orbiting planet, can directly measure features such as minimum mass, orbital period, eccentricity and the semi-major axis (Wright, 2013). Microlensing detects exoplanets by measuring the temporary brightening of a background star when a foreground star-planet system passes in front of it (Tsapras, 2018). This method directly measures parameters such as Planet-to-Star Mass Ratio, Einstein Radius and Projected Separation (Gaudi, 2012).

Each discovery method produces different parameters, resulting in difficulties for a comprehensive analysis. Sophisticated data processing is required along with normalising techniques to account for the lack of completeness and variation of data (Khan and Khan, 2024). A study done by Lalande, Tasker and Doya (2024) estimated the use of machine learning on the incomplete NASA Exoplanet Archive dataset. They wanted to see if a variable with more than 70% of no measured values could be accurately estimated using different algorithms. The study concluded that they recommended the kNN×KDE and MissForest algorithms as they showed the best level of performance out of the five machine learning algorithms. The study focused on the estimation of planet mass, however, states that the algorithms could impute any missing property present in the dataset. Mass was able to be estimated regardless of what properties had measured values. With estimates of missing values, analysis can be done with all 5,251 planet discoveries instead of a 550-planet subset.

2.2 - Exoplanet Classification

Previous research has shown how exoplanet detection can be classified into different types based on the available variables from the discovery method. An example is Weiss & Marcy (2014), who analysed 65 planets in their study. They found a strong correlation between planet mass and radius, suggesting a mass-radius relation formula for predicting exoplanet composition. The study identified a transition point at 1.6 Radii with planets being smaller than the value likely to be mostly rocky and above the value likely to be more volatile-rich. This is backed by a later study from Rogers (2015), who used statistical analysis to define a radius threshold for rocky planets. The study concluded that planets with a radius larger than 1.6R have gaseous envelopes, making them unlikely to be fully rocky. Therefore, planets below this threshold are likely terrestrial types. Lopez and Fortney (2014) looked at mass-radius relationships for sub-Neptune exoplanets. They concluded that planets between 2.5–6R belong to the Neptunian classification type. Buchhave *et al.* (2014) identified through their study that planets above 2.5R are mostly Neptunian and Gas Giant types.

Chen and Kipping (2017) used statistical modelling to predict mass-radius relations for exoplanets. They developed a probabilistic model and were able to classify exoplanets based on the available data. These four categories were Terran worlds, Neptunian worlds, Jovian worlds and Stellar worlds. When comparing this to NASA's category system, Terran worlds can be compared with Terrestrial, Neptunian Worlds with Neptunian, Jovian Worlds with Gas Giants and Stellar Worlds having no direct equivalent. This study points out that Super-Earths may be a misleading classification as these planets may be rarer than first thought.

2.3 - CRISP-DM Methodology

Data mining is the process of finding patterns, relationships and correlations from large datasets by using machine learning and other techniques. The goal is to extract useful information from raw data to allow for decision-making, predictions and classifications

(Han, Kamber and Pei, 2011). Process models like CRISP-DM and the Analytics Life Cycle provide a methodology for achieving data mining projects.

The Cross-Industry Standard Process for Data Mining (CRISP-DM) is a data mining model that data science teams use to complete projects. It consists of six iterative phases that are to be planned and followed. Business understanding is the first stage, which is when the situation should be assessed to get an overview of the required resources, and the data mining goal should be explained. Data understanding follows, which is the collection of data sources and exploring it whilst checking for data quality. Data preparation is the cleaning of data to ensure analysis is accurate. This can include removing duplicate rows or null values and standardising data values. Modelling is the phase where the analysis technique is carried out. The best technique for answering the business problem should be used, in which specific parameters are set. The Evaluation stage is where results are checked against the business objectives, and finally the deployment phase is the monitoring and maintenance of the results in a real-world setting (Schröer et al., 2021).

Previous studies related to this project have used the CRISP-DM model. A study by Rajput (2024) presented a machine learning-based approach to successfully classify exoplanets from the Kepler cumulative object of interest data obtained from NASA. The study used a data mining approach, including data preparation and processing, to find accurate classifications of exoplanets. Other studies also demonstrate the application of structured methodologies in this field (Malik, Moster and Obermeier, 2022). For this project, CRISP-DM will be applied to ensure the objectives are accurately answered through the analysis of clean data.

2.4 - Machine Learning

Machine Learning is a subcategory of artificial intelligence that allows systems to learn patterns from data. These algorithms can analyse large and complex datasets for clustering,

correlation, prediction, and more. Industries have turned to machine learning for accurate reporting and decision making, becoming one of the fastest growing areas within computer science (Zaki and Meira, 2021). There is no one-size-fits-all algorithm best suited to solve a problem, as it will depend on the objectives and dataset (Mahesh, 2020). Machine learning is said to have emerged as an important tool in offering an innovative way to analyse the large datasets generated by telescopes (Smith, 2022). It can be used for exoplanet identification and classification by combining computer programming and statistical theory to construct models (Alpaydin, 2014). Deep learning is another form of machine learning which has been used in the form of a neural network model to scour archived data to identify planets using transit events which had gone unnoticed by other researchers (Shallue and Vanderberg, 2011).

Machine learning algorithms can be divided into four major types: supervised, unsupervised, semi-supervised, and reinforcement learning (Zaki and Meira, 2021). The distinction between supervised and unsupervised machine learning must be considered before choosing a model technique for analysis.

2.4.1 - Supervised Machine Learning

Supervised machine learning is an algorithm which needs external assistance to train and learn the data (Mahesh, 2020). It is common in classification problems because the goal is regularly to get the computer to learn a classification system that has been manually created (Taiwo, 2010). Data is split into training and testing data, and the training data is trained until the desired level of accuracy is achieved. This is then tested and evaluated on the test dataset. The highest level of accuracy will suggest the model is best suited to analyse the data and produce the most reliable results for deployment. Algorithms which are under the supervised learning type include classification models like decision trees, random forest, Support Vector Machine and k-Nearest Neighbours.

Examples of studies which have used supervised machine learning with exoplanet datasets include a study by Sturrock *et al.* (2019) who focused on using machine learning for the classification of discoveries as exoplanets or “false positives”. Binary classification of exoplanets as “FALSE POSITIVE” or “CONFIRMED” were tested through comparing random forest, Support Vector Machine and k-Nearest Neighbours models. After testing for accuracy, recall, and precision to arrive at the superior model, the Random Forest model was selected for the classification analysis as it showed the best numbers in the training metrics. Another study implemented logistical regression, decision tree, and neural network models to find the probability of the existence of an exoplanet candidate (Saha, 2019). The analysis concluded that the Decision Tree model performed best for prediction accuracy, statistical significance, and execution time of classifying Kepler exoplanet candidates, with an accuracy of 98.53%.

2.4.2 - Unsupervised Machine Learning

Unsupervised machine learning is when a model algorithm is left on its own to discover and present structures in the data. Only input data is used for the algorithm to learn from, as the aim is to find the regularities in the input. This is referred to as density estimation, allowing the identification of regions of high and low data concentration without prior labels (Alpaydin, 2014). Clustering is a type of density estimation where the aim is to find groupings of input similar in their attributes. Examples of these methods include K-Means Clustering, Hierarchical Clustering and Principal Component Analysis.

Previous studies have used unsupervised machine learning methods along with exoplanet datasets. A study by Jin *et al.* (2022) looked at clustering planets like Earth based on variables available in the NASA Exoplanet Archive dataset. All exoplanets were divided into 100 clusters where the cluster which contained Earth were said to be the most likely to be habitable. This cluster had a total of 39 exoplanets. Another study combined the use of Principal Component Analysis (PCA) and K-Means Clustering to optimise exoplanet

atmosphere retrieval. PCA was used to compress the spectral information down from 461 wavelength bins to just 10 PCA components, with minimal loss of information and used a k-means clustering algorithm for unsupervised classification of the models. This approach successfully categorised transmission spectra into distinct clusters, allowing for more efficient parameter retrieval, showing a high classification accuracy of 98% and low noise of up to 10% (Hayes *et al.*, 2020).

2.4.3 - Reinforcement Machine Learning

Reinforcement Machine Learning is an algorithm that enables machines to automatically evaluate optimal behaviour. The algorithm relies on rewards and penalties and aims to leverage insights to enhance rewards and minimise risks. This approach is primarily used for training AI models to create automation or optimise the efficiency of complex systems, such as robotics (Sarker, 2021).

Examples include AlphaZero and AlphaGo, which are reinforcement learning techniques that take up the model-based approach. This performs actions and observes the results, creating a predictive model of the environment to optimise decision-making through self-play and iterative learning (Silva *et al.*, 2016). Other techniques involve a model-free approach, which does not use the reward function. Q-learning, Deep Q Networks and Monte Carlo Control are examples of these types of algorithms which learn optimal policies through direct interaction with the environment (Kaelbling *et al.*, 1996).

2.5 - Machine Learning Algorithms

2.5.1 - K-Means Clustering

The K-Means Clustering Model is an unsupervised algorithm which is used to cluster data points. This is one of the simplest algorithms for clustering analysis, which defines K centres for each cluster called centroids. The K-Means Cluster is determined by the formula:

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Figure 2: K-means Cluster Formula for the K-Means Clustering Algorithm

where μ_i is the mean (centroid) of points in C_i , k is the number of clusters, x is a data point in cluster C_i , and $\|x - \mu_i\|^2$ is the squared Euclidean distance between a data point and its assigned cluster centroid. The optimum number of clusters can be determined through the elbow method, which identifies the optimal point for determining the k value at the point in which there is a significant change in the Within-Cluster Sum of Squares (WCSS) value, creating an angle (Marutho *et al.*, 2018). The phases of the algorithm are initialisation, assignment, recalculation, and repetition. Initialisation is when the initial cluster centroids are randomly selected. Assignment is when metrics such as the Euclidean Distance assign each data point to the nearest cluster centroid. Recalculation is when each cluster's centroid is calculated again as the mean of all assigned points, and repetition is the redoing of the previous steps until centroids stop changing significantly (Jin and Han, 2011). No target variable is assigned as the algorithm autonomously finds clusters based on the available features in the data. K-means clustering is useful for investigating large datasets and finding the key similarities and differences between the data points.

K-means separates boundaries between clusters linearly; the Kernel K-means algorithm allows the extraction of non-linear boundaries between clusters. This algorithm can capture

complex clusters where the chosen dataset is irregular by incorporating kernel functions. As much real-world data is often non-linearly separable, feature mapping can be applied to the machine learning algorithm to map data to a higher-dimensional space. This project will use the K-Means algorithm to automatically classify exoplanets based on the available features in the dataset.

2.5.2 - Support Vector Machine (SVM)

A Support Vector Machine (SVM) learning algorithm is a hyperplane-separating classifier. This supervised machine learning technique divides the labelled training data between a line dividing hyperplane into two (Singh and Kumbhare, 2022). There is a low risk of overfitting due to its relative simplicity, its ability to be applied to non-linear data, and its robustness to outliers (Saha, 2019). A hyperplane is defined as:

$$w^T x + b$$

Figure 3: Hyperplane Formula for the SVM Algorithm

Where w^T is the weight vector transpose and b is the bias. D-dimensional real space is the number of independent variables used to describe a data point which is defined by the equation:

$$x \in R^d$$

Figure 4: D-dimensional Real Space Formula for the SVM Algorithm

Where x represents a vector, R represents the set of all real numbers, and d indicates that the vector has a certain number of features for analysis (Singh and Kumbhare, 2022). The margin is the distance between the hyperplane and its closest data point. Training data that

is linearly separable can select two parallel hyperplanes which separate two classes of data, making the distance as large as possible. When the data is not linearly separable, a soft margin can be created. This allows some constraints to be violated and cross the margin whilst keeping it as large as possible (Saini, 2024). Complex patterns can be captured using Kernel SVM, which is an extension of SVM. This maps the original features into a higher-dimensional space where the kernel function is defined by:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Figure 5: Kernel Function Formula for the SVM Algorithm

Where $\mathbf{x}_i$ and $\mathbf{x}_j$ are the input data points and $\phi(\mathbf{x})$ is a transformation function mapping data into a higher-dimensional space. Choosing the right Kernel function is important as it affects the accuracy of the analysis. The selection depends on the complexity and distribution of the data (Chatterjee *et al*, 2018).

SVM is a binary classifier however, a multi-class classification strategy can still be applied. Multi-class classification with SVM is achievable using either the One- vs.-One (OvO) or One- vs.-All (OvA) strategy (Krebs *et al.*, 2024). The OvO algorithm involves training a separate classifier for each pair of classes, resulting in many binary classifiers that collectively handle the multi-class problem. The OvA method trains a single classifier per class, distinguishing one class from the others. A study carried out by Krebs *et al.* (2024) found that the OvO model outperformed the OvA model in terms of accuracy and F1-score whilst the models showed similar precision results. The OvO model differentiated similar classes better and showed fewer misclassifications, therefore performing better overall. In this project, SVM will be used with the aim of classifying exoplanets into four groups. The OvO or OvA method will be implemented in attempts to use SVM for multi-class classification and the Kernel function will be used for the non-linear data.

2.5.3 - Decision Trees

Decision Trees are a versatile supervised machine learning algorithm that can perform classification, regression and multi-label tasks (Géron, 2022). Complex datasets can be fitted with this model by capturing non-linear relationships through hierarchical decision rules.

The Classification and Regression Tree (CART) algorithm splits the training set into two subsets and then recursively until it reaches the maximum depth. The training algorithms compare all features at each node to reach an optimum split, which can be executed in one of three ways depending on the categorical nature of the problem. A node represents a point where the dataset is split based on a specific feature to create branches, leading to further splits or final classification (Rokach and Maimon, 2008). A hyperplane specifies the split point as it splits data into two half-spaces (Zaki and Meira, 2020).

The misclassification rate, Gini index and entropy criterion are metrics used in decision tree algorithms to measure the impurity of a node and determine the best feature splits for classification. Entropy measures the amount of uncertainty in a system, where if a partition has a low entropy, it is relatively pure. A higher entropy shows the labels are mixed (Zaki and Meira, 2020). This is calculated by:

$$Entropy = \sum_{i=1}^c -P_i * \log_2(P_i)$$

Figure 6: Entropy Formula for Decision Tree Algorithm

Where c is the number of classes and p_i is the probability of a sample belonging to a class i . The $\log_2$ in the formula measures the amount of information required to encode a class. An entropy score of zero would indicate a region is pure and has points from the same class.

The Gini index can also gauge the purity of a split in the decision tree algorithm. It is calculated by:

$$Gini = 1 - \sum_j p_j^2$$

Figure 7: Gini Index Formula for Decision Tree Algorithms

Pruning is the deletion of any unnecessary nodes by selecting the number of leaves in the tree, preventing overfitting in the analysis (Lindholm et al., 2022). Pruning can be carried out through pre-pruning, which is forward-stopping or post-pruning, which is the forward-stopping of branches. The Decision Trees Machine Learning model will be used in this study as a supervised method to classify exoplanets based on the features available in the dataset.

2.5.4 - K-Nearest Neighbour (KNN)

The K-Nearest Neighbour Machine Learning is a supervised model that can be used for regression and classification. This is deemed a simple algorithm which does not make any assumptions or learn during the training phase but instead learns during the testing phase (Lindholm et al., 2022). KNN can be calculated using the Euclidean distance formula, which has a mix of efficiency and ease (Kataria and Singh, 2013). The Euclidean distance is calculated by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Figure 8: The Euclidean Distance Formula for the KNN Algorithm

Where (x_1, y_1) represents the first point and (x_2, y_2) represents the second point of the coordinates in a 2D plane. This is essentially applying the Pythagorean Theorem to measure the true straight-line distance between two points in Euclidean space (Deisenroth *et al.*, 2020). Other distance metrics such as the Minkowski, Manhattan, Cosine and Jaccard distances can be used to identify the nearest neighbour of a data point. The Euclidean distance can also be used in a higher-dimensional space by adding more squared differences to each coordinate. If the KNN algorithm is analysing more than two variables in a dataset, a higher dimension would be required, making the algorithm compute similarly based on all variables.

The number of k must be considered when making predictions using KNN. This is a choice left to the user and is not determined by the model itself. Selecting the number of k is a trade-off between flexibility and rigidity, which can lead to meaningless classifiers if the number is too big or too small (Lindholm *et al.*, 2022). A sweet spot should be the aim, which allows the classifier to generalise the best. The re-scale of the input variables is important for this model as different ranges can lead to some being considered more important than others during analysis. This is achieved through standardisation which uses the mean and standard deviation in the training data for each input variable (Géron, 2022). Min-max scaling is the simplest, where each variable subtracts the minimum value and divides it by the difference between the minimum and maximum values. Standardisation subtracts the mean value and then divides the results by the standard deviation. This is less affected by outliers compared to the min-max method, however does not restrict values to a specific range (Géron, 2022).

2.6 - Model Comparison

Model comparison can be used to find the best machine-learning technique for analysing the dataset. First, machine learning models have advantages and disadvantages, depending on the objectives and type of data available. Second, results can be compared for accuracy post-analysis using various model comparison techniques.

When looking at clustering-type machine learning methods, pros and cons can be identified. The K-Means unsupervised clustering machine learning model is useful for the investigation of large datasets. It does not need a target variable to be identified as all variables are treated equally and allow all types of input variables. The algorithm is a relatively simple process to understand and execute, creating useful clusters based on similar values in the input variables. Difficulties faced when applying this method include choosing the right distance measurement and the choice of k for the number of clusters. Not picking the optimum number of k can result in the data being difficult to interpret and even making the analyses meaningless. The model can also struggle with noisy data, such as outliers (Zaki and Meira, 2020).

When looking at decision trees, it is a simple algorithm to run which are robust to outliers and can deal well with missing values in input variables. However, decision trees can become unstable and sometimes struggle to deal with continuous input variables (Jijo and Abdulazeez, 2021). SVM works better when the data is linear and is not sensitive to outliers. One thing which SVM struggles with is showing accurate results with big datasets, with training time largely increasing, making it computationally expensive. This algorithm is also sensitive to outliers, making it weak when dealing with noisy data (Saini, 2024). KNN is a model which is easy to implement. No training is required in the algorithm, and it also works well with non-parametric data. Like SVM, this model can become slow when large datasets are used and is highly dependent on the correct number of k being selected which can be difficult to decide (Witten *et al.*, 2016).

It can be difficult to determine which machine-learning technique is the best for completing the objectives based on the dataset. Many studies carry out multiple machine-learning techniques on the same data to compare which algorithm produces the best results in answering the objectives. Performance measures such as F1-score and Accuracy can be calculated to evaluate which model is the best. Accuracy is a simple metric, which is the

ratio of the total number of correct predictions to the total number of predictions (Huilgol, 2024). This is calculated as:

$$\text{Accuracy} = \frac{\text{True Positive} + \text{True Negative}}{(\text{True Positive} + \text{False Positive} + \text{True Negative} + \text{False Negative})}$$

Figure 9: Accuracy Formula for Model Comparison

Where True Positives (TP) are all cases in the analyses that correctly predicted positive cases, and False Positives (FP) are cases in which positive cases were incorrectly predicted. False Negatives (FN) are cases in which the positive cases are missed, and True Negatives (TN) are cases in which the analysis correctly predicts as negative.

F1-score evaluates the balance between precision and recall and is calculated by:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Figure 10: F1 Score Formula for Model Comparison

Where precision and Recall are calculated by:

$$\begin{aligned}\text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN}\end{aligned}$$

Figure 11: Precision and Recall Formula for Model Comparison

Where Precision is the measurement of how many of the predicted positive instances were correct, and Recall measures how many actual positive instances were identified correctly. Both precision and recall must be calculated for accurate model evaluation, which is referred to as the Precision/Recall Trade-off. This ensures that the model performs well within different scenarios, and focusing on one metric can lead to suboptimal outcomes. A model with high precision may lead to many TP being missed, and a model with a high recall may include a high number of FP (Huilgol, 2024).

Accuracy, Precision, Recall and F1-score can be used as metrics to measure model reliability. They each provided different numerical feedback on the machine learning model, which can be important in evaluating the overall performance. These measurements can provide a good fit for the model, allowing a balance between underfitting and overfitting and a trade-off between bias and variance (Huilgol, 2024). In this study, performance measurements will be calculated for each selected machine learning technique and compared with each other to find which algorithm best achieves the greatest fit in analysing the dataset.

3 - Methodology

3.1 - Business Understanding

The CRISP-DM methodology will be followed to help enhance the data mining approach. The aim, objectives and research question were defined during the business understanding phase, using a classification approach. A compulsory plan was made during this phase to ensure the following phases had a time frame for completion. This study aims to classify exoplanets using various machine-learning techniques. Appropriate steps must be taken during the methodology phase to achieve this, as many types of machine learning techniques can be used for analysis, such as classification, regression, association, and predictive algorithms. Machine learning methods are widely utilised in scientific research to build classifiers that separate data into multiple classes (Malik *et al.*, 2022). The goal is to determine the most effective classification model based on key planetary characteristics, employing model comparison techniques to evaluate the success of each algorithm. The algorithms should handle large, complex, and non-linear datasets to identify patterns in the data, creating clusters based on the available key features. This can be accomplished through supervised and unsupervised machine learning techniques, as labels do not have to be predefined.

A range of software can be chosen for machine learning, such as R, Java and Python. For this study, the latter will be used for multiple algorithms and compared to find the optimum method. R will be used for preprocessing structured data from the NASA Exoplanet Archive, ensuring that only relevant and clean data is used for machine learning analysis.

3.1.1 - Software

Software selection is critical for the successful execution of a data mining project. Based on the business understanding, specific software and programming languages must be selected to ensure the analysis can be carried out correctly.

Python was selected as the primary programming language due to its extensive machine-learning libraries. This programming language can manipulate, clean, visualise and process data using different libraries. Different data can work within Python, such as spreadsheet-like data or multidimensional matrices (McKinney, 2022). Essential Python libraries include NumPy, pandas, matplotlib and scikit-learn and others, which will be implemented during the analysis phase of this project.

R is a programming language designed for statistical computing, data analysis, and graphical representation. Data preparation and cleaning in R can be executed using various libraries such as Tidyverse, dplyr and tidyr. These packages can select, filter and summarise datasets so that it is understood and ready for analysis (Morrow, 2013). Certain phases of the project will incorporate R to complete various tasks, ensuring efficient data analysis and visualisation results.

Power BI is a business intelligence and data visualisation platform developed by Microsoft, used to transform raw data into valuable insights through reports and dashboards. Like Python and SQL, it can also clean, transform and model data through Power Query. Data Analysis Expressions (DAX) can be used to create calculated measures and perform complex calculations. The main use of Power BI is the visualisation capabilities, where dynamic charts, graphs, maps, and tables can be created, allowing trends and patterns to be identified. This data analysis tool will be used to visualise the results gathered from the analysis so that key points can be identified (Metre *et al.*, 2023).

3.2 - Data Understanding

The second phase of the CRISP-DM technology is to understand the data which will be analysed to achieve the aim and objectives of the project. The data source must include all the information required during the analysis phase, which data exploration can be carried out. The data source and harvesting process are mentioned in this stage and descriptive statistics are obligatory (Schröer *et al.*, 2021). The response variable and explanatory variables must be identified for supervised machine learning. The data type must also be identified. Different types of data such as continuous, ordinal or categorical respond differently to machine learning models, which means the relevant algorithm must be selected based on the data type. Visualisation techniques can be used to gather a good understanding of the overall data such as boxplots and histograms to understand the skewness and kurtosis of the data. Other statistical explorations like the mean and standard deviation help with data understanding. Finally, correlation analysis can be performed to compare explanatory variables so that insignificant variables are left out.

For this project, the dataset source was gathered from the NASA Exoplanet Archive, which provides publicly available and peer-reviewed data on confirmed exoplanets. This includes over 5,000 rows of exoplanets with variables necessary for classification machine learning model techniques (NASA Exoplanet Science Institute, 2024).

3.3 - Data Preparation

During the data preparation phase, the selection, cleaning, and transformation of the raw dataset are carried out to ensure its suitability for classification in machine learning. This is one of the most time-consuming phases of data mining and is crucial for the project's success, taking between 50% and 70% of the overall time. Input and output data should be described along with the methods and approaches (Schröer *et al.*, 2021).

Cleaning data refers to listing the variables selected based on the data understanding, explaining the data types, and addressing any missing values or outliers. There are different solutions to handling incomplete data. Typically, missing data is handled by eliminating any cases with missing values using a technique called listwise deletion. This method removes any row containing at least one missing value. Other methods can be used to prevent a reduced sample size of data such as the mean substitution, regression, expectation-maximisation and mean likelihood method (Rubin *et al.*, 2007). These are imputation methods which replace missing values with reasonable substitutes. Mean substitution handles missing data by replacing the missing values with the mean value of the non-missing rows in the same variable. The regression method predicts the missing value using a regression model based on other variables in the dataset. The expectation-maximisation method is a statistical technique that estimates missing data. It maximises the likelihood function based on observed data to replace missing values. The maximum likelihood approach uses parameter values with the highest probability possibility using a likelihood function (Rubin *et al.*, 2007). These imputation methods perform differently under various circumstances and should be chosen based on which will have the greater accuracy. Deletion through the listwise deletion method is necessary when missing values present in a dataset are less than 10 (Rubin *et al.*, 2007). Expectation-maximisation is most effective when missing values are moderate to large, and it can provide consistent estimates of values when incomplete data is up to 50% (Choi *et al.*, 2004). Previous studies have concluded that this was the most accurate method at all levels of missing data and that the mean substitution was inferior to it and to the regression method (Wothke, 1998). As the NASA Exoplanet Archive dataset contains a high number of missing values, imputation methods will be used during data preparation to replace the missing values with accurate prediction measurements. This will maximise the number of rows used for classification analysis, allowing for a greater sample size.

Data transformation is another part of the data preparation phase. Feature scaling is important during data preparation to transform variables and allow them to exhibit

properties of a normal distribution. Most machine learning algorithms perform better when the input variables have similar numerical values, preventing variables with smaller scales from being dominated by those with much higher scales. Without scaling, models will become biased towards a certain variable. Two common ways are used to get all attributes to the same scale: min-max scaling and standardisation (Géron, 2022).

Min-max scaling is the simplest, where for each row, the values are rescaled so they have a range of 0 to 1. The formula for this is:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Figure 12: Equation for Min-max Scaling

Where the minimum value is subtracted and divided by the difference between the minimum and maximum value. Standardisation is calculated with the formula:

$$X_{scaled} = \frac{X - X_{mean}}{X_{stddev}}$$

Figure 13: Formula for Standardisation

Where the mean value is subtracted and then divided by the standard deviation. This formula does not restrict values to a specific range like Min-max scaling. It must be noted that when variables are skewed or have high kurtosis, both data transformation methods will condense the values into small ranges, potentially negatively affecting the machine learning model. Therefore, it is important to shrink skewness and kurtosis before transformation, making the distribution as normal as possible. If there is a long and heavy tail in the variable, then approaches such as Log Transformation or bucketising may be used (Géron, 2022).

3.4 - Modelling

The modelling stage of CRISP-DM is the analysis of the dataset, where there are many approaches available to use depending on the purpose of the project. Choosing the appropriate model depends on the type of data used and the project's objectives. For this stage, classification machine learning models will be implemented to find similarities in the NASA Exoplanet Archive dataset and create clusters. Other types of analysis have been carried out on this dataset such as predictive and deep-learning analysis (Lalande, Tasker, and Doya, 2024) (Valizadegan et al., 2022).

Data partition is important during this stage of data mining, which is the selection and training of a model. This will optimise the performance of the model, allowing the algorithm to learn from a part of the data whilst being tested on unseen data. Training data is used to build the model and the test data assesses the model performance. The data partitioning can be done at different splits, where the most common split for machine learning when data is fairly sized is 80% training and 20% testing. This ratio prevents overfitting of the data, allowing the model to generalise the dataset well (Gholamy et al., 2018).

The training data can be further partitioned into a validation set, which greatly refines the data. Cross-validation is a technique which splits the training set into a smaller training set and a validation set. The model trains against the smaller training set and then evaluates its performance against the validation set. Another cross-validation method includes k-folds, which split the training set into ten nonoverlapping subsets called folds creating ten evaluation scores (Géron, 2022).

The right statistical models should be chosen for this stage based on the type of data provided and the aims and objectives of the project. Necessary parameters should be implemented in each machine learning model to maximise the accuracy of the algorithm.

Classification and clustering machine learning models will be used, including the k-means, decision tree, SVM and KNN models.

3.5 - Evaluation

Once the models have analysed the data, their performance can be compared in the evaluation phase. The performance will be considered based on statistical results such as the F1-score, precision and recall scores. Figures such as the confusion matrix and ROC curve can be used to visualise the performance and accuracy of each model. The receiver operating characteristic (ROC) curve is a visualisation tool that plots the recall against the false positive rate. The higher the recall, means more false positives are produced from the classification model. A good classifier will show a line as far away into the top-left corner of the figure (Géron, 2022). The evaluation of the models should also be based on their ability to answer the project's aims and objectives.

3.6 - Deployment

The final stage of the CRISP-DM methodology in this project is the deployment stage. This is the implementation of strategic actions produced from the analysis and the post-maintenance. A lot of studies do not foresee a deployment phase and stop at the evaluation stage (Schröerab *et al.*, 2022). This is because a lot of studies provide a concept or compare models, which means deployment is not the goal of their study. This is the same for this project, as one of the objectives is to compare different classification models to find the most accurate algorithm for the classification of exoplanets. Therefore, the deployment phase for this project will not be needed, and the methodology will stop at the evaluation stage.

4 - Analysis

4.1 - Data Understanding

The exoplanet dataset consisted of 122 variables and 38,063 rows, of which 29.7% of cells contained null values. R library packages including tidyverse, dplyr and readr were used to identify the data types, provide descriptive statistics, find correlations between variables and use visualisation techniques to understand the data. Different discovery methods have been used to detect exoplanets, resulting in variations in which planetary variables are measurable and thus causing a high number of null values. As the aim of this project is to use machine learning to classify exoplanets, only relevant variables in the dataset were used, and many were dropped. Of the 122 Variables, 16 were kept as seen in Table 1. These were selected based on their description and previous research related to the project. With these variables, it is hypothesised that exoplanets can be successfully classified into different groups: Gas Giant, Neptunian, Super-Earth, and Terrestrial.

Table 1: Variables selected for Machine Learning Classification

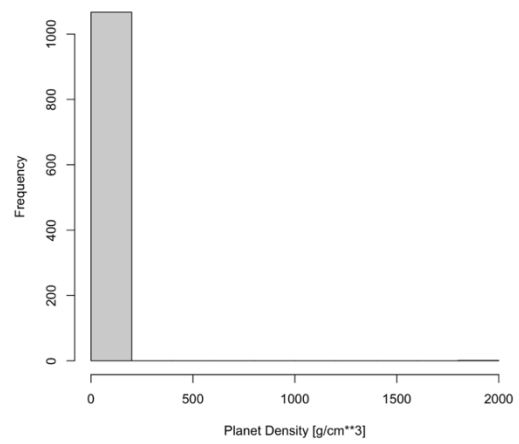
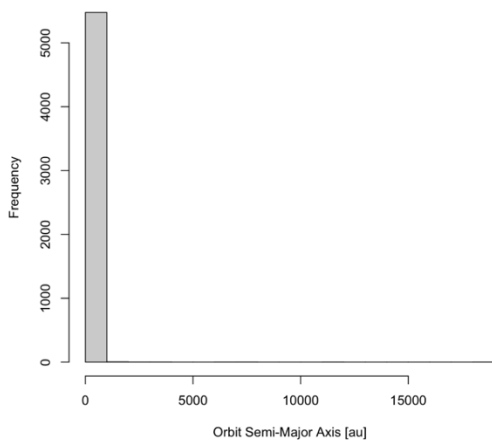
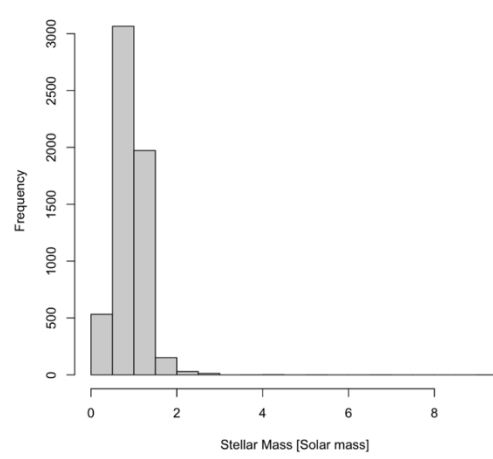
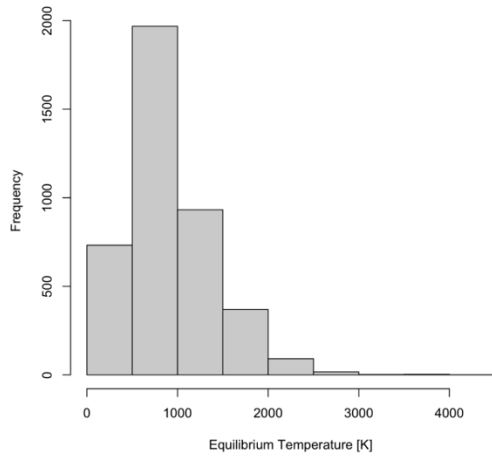
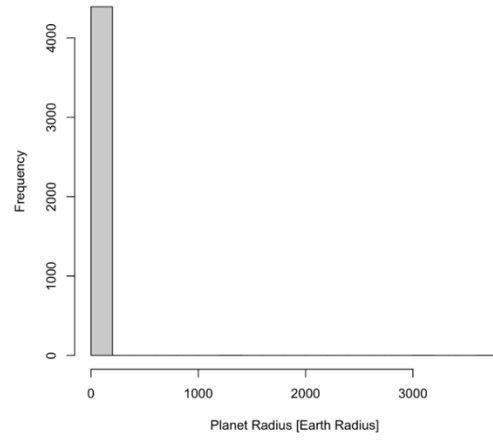
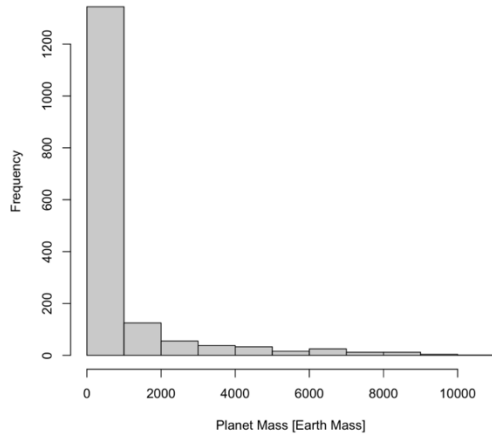
Column Name	Name	Variable Type	Description
pl_orbper	Orbital Period [days]	Int	The time it takes for an exoplanet to complete one full orbit around its star
pl_orbsmax	Orbit Semi-Major Axis [au]	Int	The average distance between the exoplanet and its star, measured in astronomical units (AU)
pl_rade	Planet Radius [Earth Radius]	Int	The radius of the exoplanet expressed as a multiple of Earth's radius
pl_masse	Planet Mass [Earth Mass]	Int	The mass of the exoplanet compared to Earth's mass
pl_dens	Planet Density [g/cm**3]	Int	The average density of the planet, useful for inferring its composition
pl_orbeccen	Eccentricity	Int	A measure of how elongated an exoplanet's orbit is, ranging from 0 (circular) to 1 (parabolic)

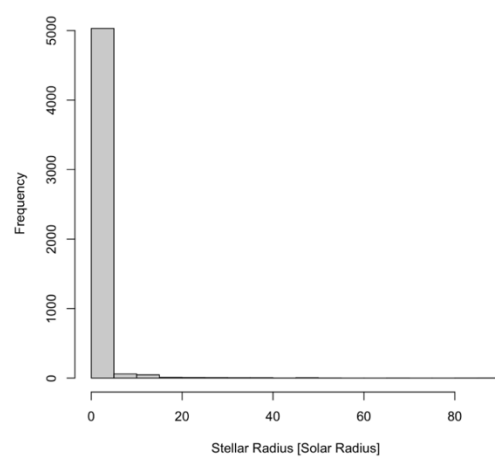
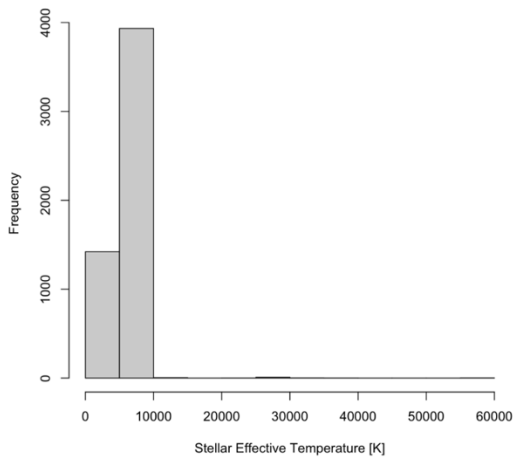
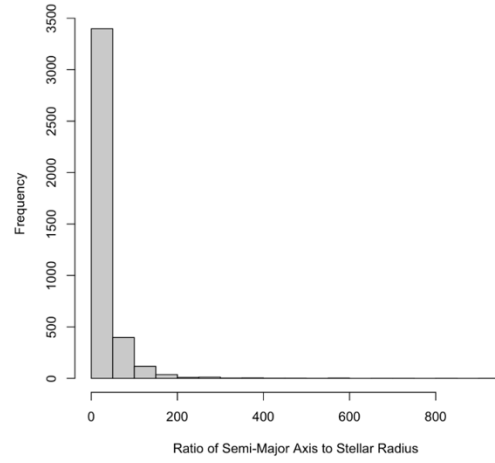
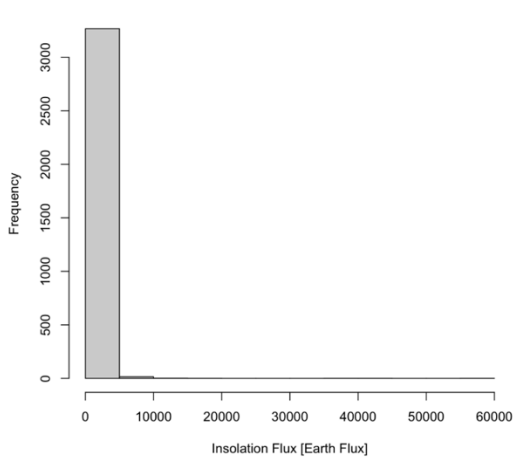
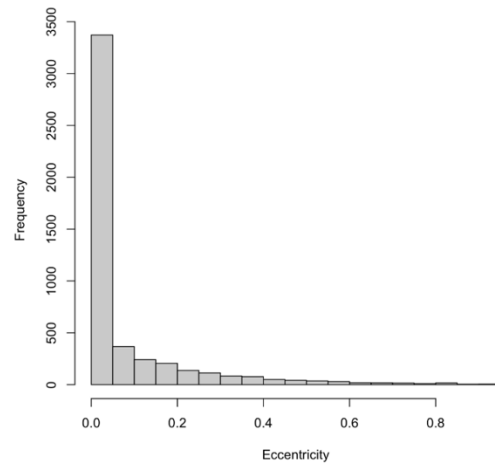
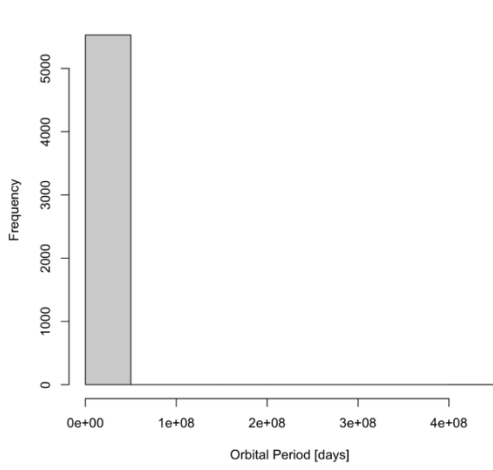
pl_insol	Insolation Flux [Earth Flux]	Int	The amount of stellar energy received by the planet, relative to what Earth receives from the Sun
pl_eqt	Equilibrium Temperature [K]	Int	The estimated surface temperature of the planet assuming no atmosphere and perfect energy balance
pl_ratdor	Ratio of Semi-Major Axis to Stellar Radius	Int	The size of the planet's orbit relative to the radius of its host star
st_teff	Stellar Effective Temperature [K]	Int	The surface temperature of the star, indicating how hot it is
st_rad	Stellar Radius [Solar Radius]	Int	The radius of the host star, given in units of the Sun's radius
st_mass	Stellar Mass [Solar mass]	Int	The mass of the star, expressed as a fraction or multiple of the Sun's mass
st_lum	Stellar Luminosity [log(Solar)]	Int	The logarithm of the star's brightness relative to the Sun
st_age	Stellar Age [Gyr]	Int	The age of the star in billions of years (gigayears)
st_dens	Stellar Density [g/cm**3]	Int	The average density of the star, useful for inferring its size and stage of evolution
Pl_name	Planet Name	String	Name of Exoplanet

When checking for duplicates, it was found that there were no rows containing the same data across all variables. However, the planet name column contained the same names of exoplanets multiple times. These were removed, with the unique row of each planet name being kept based on the least null values it contained, and therefore changing the number of rows from 38,063 to 5,819. For example, exoplanet 14 Her b appeared eight times in the dataset. Once the duplicates were the unique row which remained had seven null values. It was verified that all measurements were numerical for classification analysis. Planet Name was later dropped before the machine learning procedure.

Box plots and histograms were produced to view how the data is distributed for each variable prior to any transformation. Anomalies and the symmetry of data can be seen in box plots when the data is continuous. Histograms show the distribution of values in a variable and show the amount of skewness and kurtosis. Datasets containing variables with many outliers, asymmetry and high levels of skewness and kurtosis make machine learning

algorithms struggle to find clear patterns, which often leads to less accurate predictions. Therefore, it was important to assess each variable during this stage, before using the machine learning models for classification.





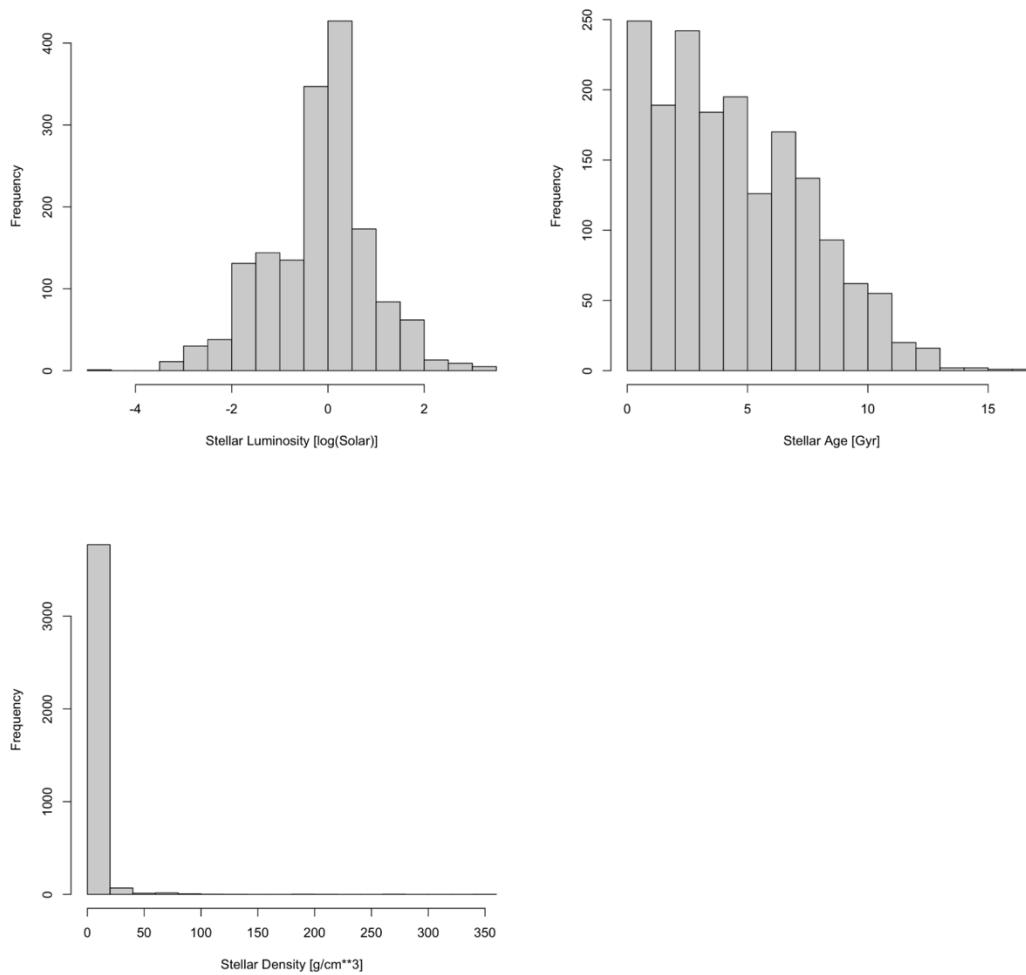
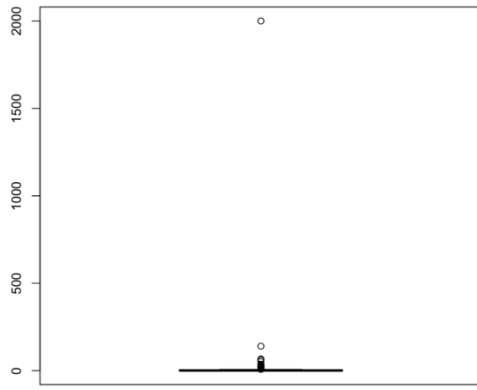
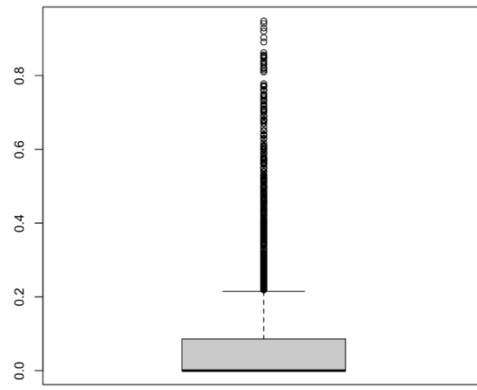


Figure 14: Histogram of the distribution of data for each selected variable

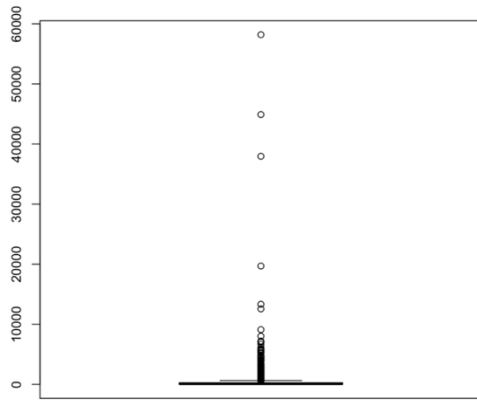
The histograms in Figure 14 mostly show an uneven data distribution in the variables. Measurements such as stellar density, stellar radius, eccentricity, orbital period, and insolation flux are heavily skewed to the right, showing that the mean is greater than the median. Most of the clusters of data are at the lower range of values, with a few being at the other end of the tail. Stellar Luminosity is more evenly distributed, however shows positive kurtosis as the peak of the histogram is high. This suggests there are extreme values and outliers for the high values of the variable. Outliers can also be seen in Figure 15 with the box plot results.



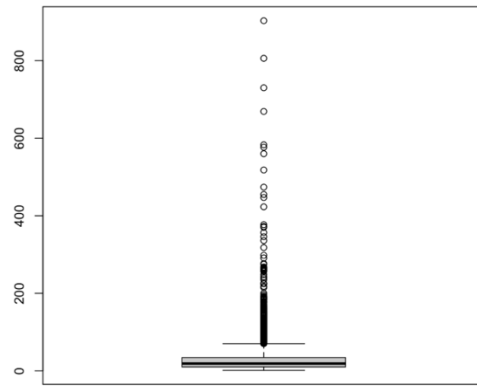
Planet Density [g/cm**3]



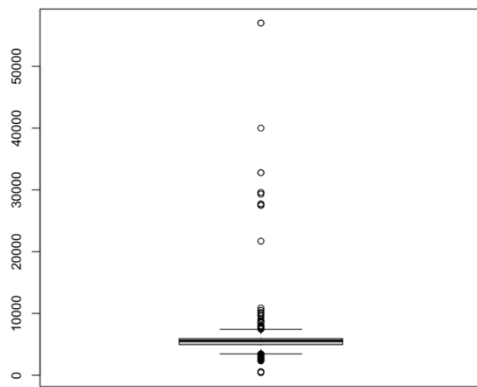
Eccentricity



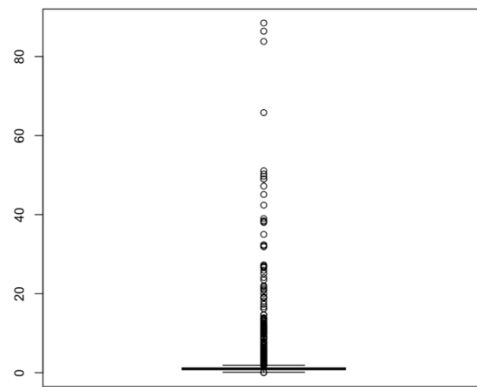
Insolation Flux [Earth Flux]



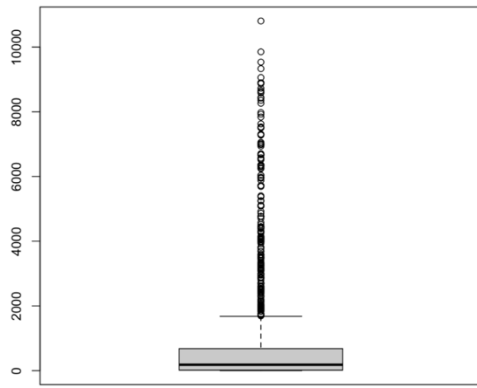
Ratio of Semi-Major Axis to Stellar Radius



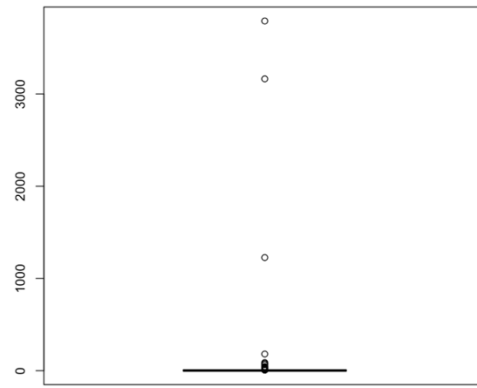
Stellar Effective Temperature [K]



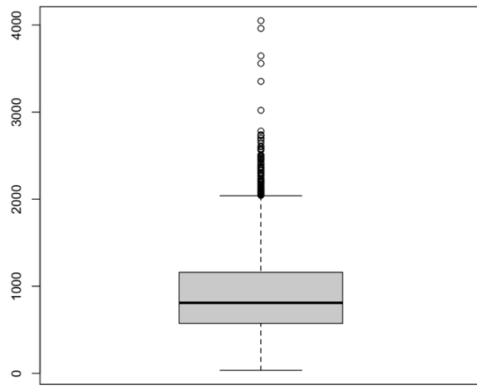
Stellar Radius [Solar Radius]



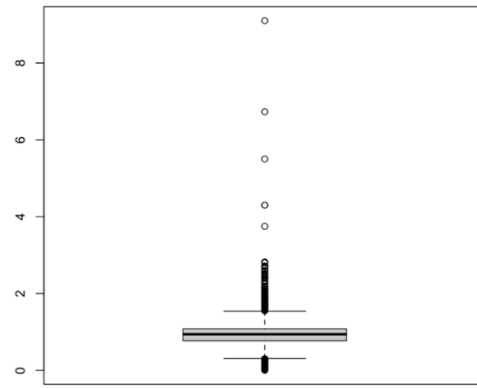
Planet Mass [Earth Mass]



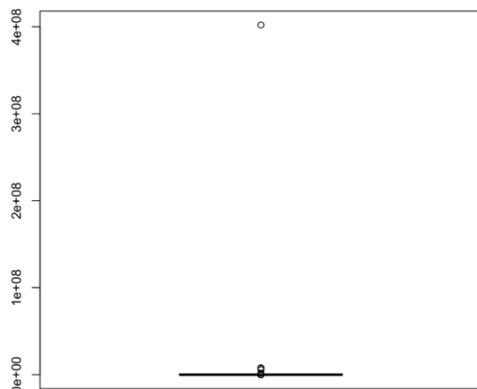
Planet Radius [Earth Radius]



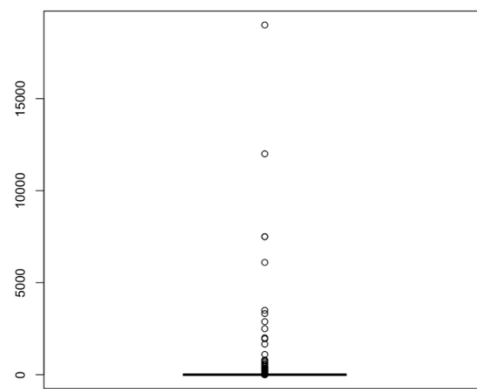
Equilibrium Temperature [K]



Stellar Mass [Solar mass]



Orbital Period [days]



Orbit Semi-Major Axis [au]

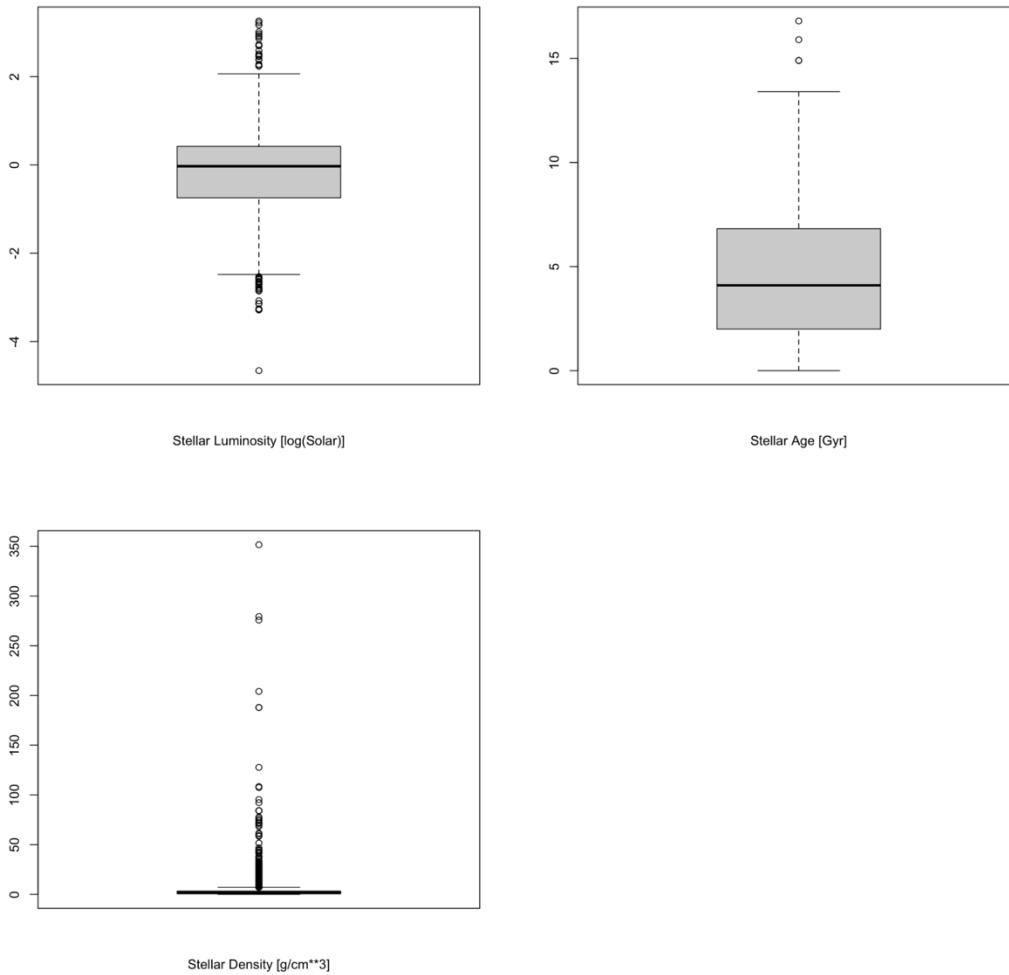


Figure 15: Box Plot of the distribution of data for each selected variable

All variables had extreme values present in their data based on the box plots, suggesting there are outliers which should be removed before classification. Based on the visualisation techniques, various data cleaning and transformation techniques were required to make the dataset suitable and reliable for accurate classification machine learning. Descriptive statistics show minimum, maximum, mean and median values of each variable along with the number of null values. All variables contained missing values, were to be sorted based on the number of missing values present. Percentages of missing values were calculated to assess which method should be used to replace these null values.

Table 2: Percentage of Missing Values in the Dataset

Variable Name	Missing Values	% of NA
pl_orbper	289	5.0
pl_orbsmax	329	5.7
pl_rade	1423	24.5
pl_masse	4154	71.4
pl_dens	4751	81.6
pl_orbeccen	979	16.8
pl_insol	2531	43.5
pl_eqt	1704	29.3
pl_ratdor	1828	31.4
st_teff	444	7.6
st_rad	635	10.9
st_mass	49	0.8
st_lum	4209	72.3
st_age	4075	70
st_dens	1937	33.3

4.2 - Data Preparation

4.2.1 - Handling of Missing Data

Variables which contained less than 10% of null values were kept and handled by imputing the median value. This is due to the low number of null values present in these variables, creating a low impact on the final classification results whilst keeping all available rows. Variables were checked to see which were statistically related to each other through correlation analysis, so regression imputation could be executed using variables with the highest correlation. The relation between variables can also be seen through scatter plots, showing a positive, negative or non-linear relationship.

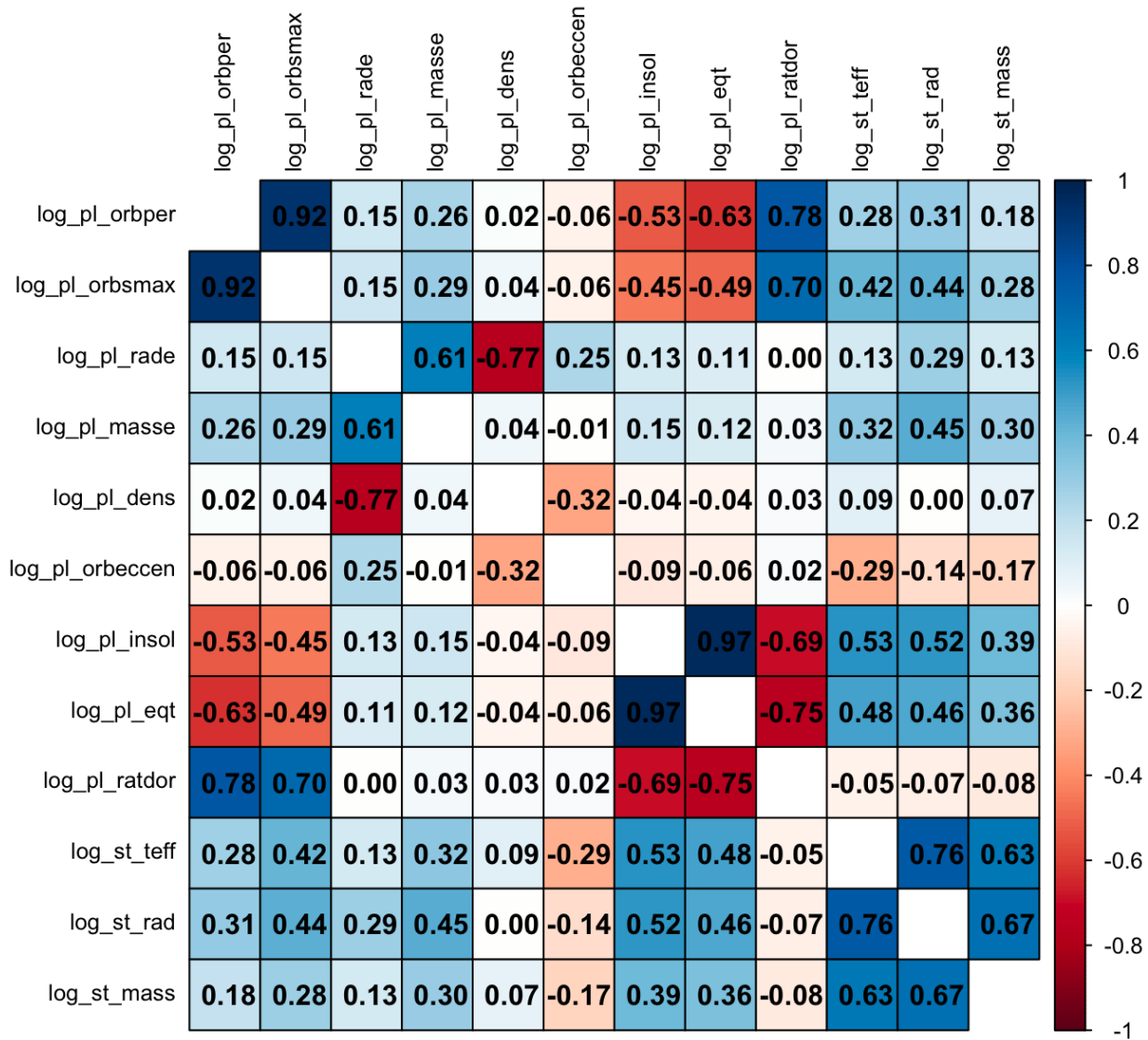


Figure 16: Correlation Matrix Between Variables

Planet mass was shown to have a positive correlation with planet radius of 0.61 and stellar radius of 0.45. Insolation Flux had a strong positive correlation with the Equilibrium temperature variable at 0.97 and a negative correlation with the planet radius at -0.69. The Ratio of Semi-Major Axis to Stellar Radius variable had a positive correlation of 0.78 with Orbital Period. Planet Density is highly correlated to Planet Radius at a negative value of 0.77.

The results from the correlation matrix allow for variables with missing data to be imputed based on other strongly correlated variables, ensuring the given values are accurate.

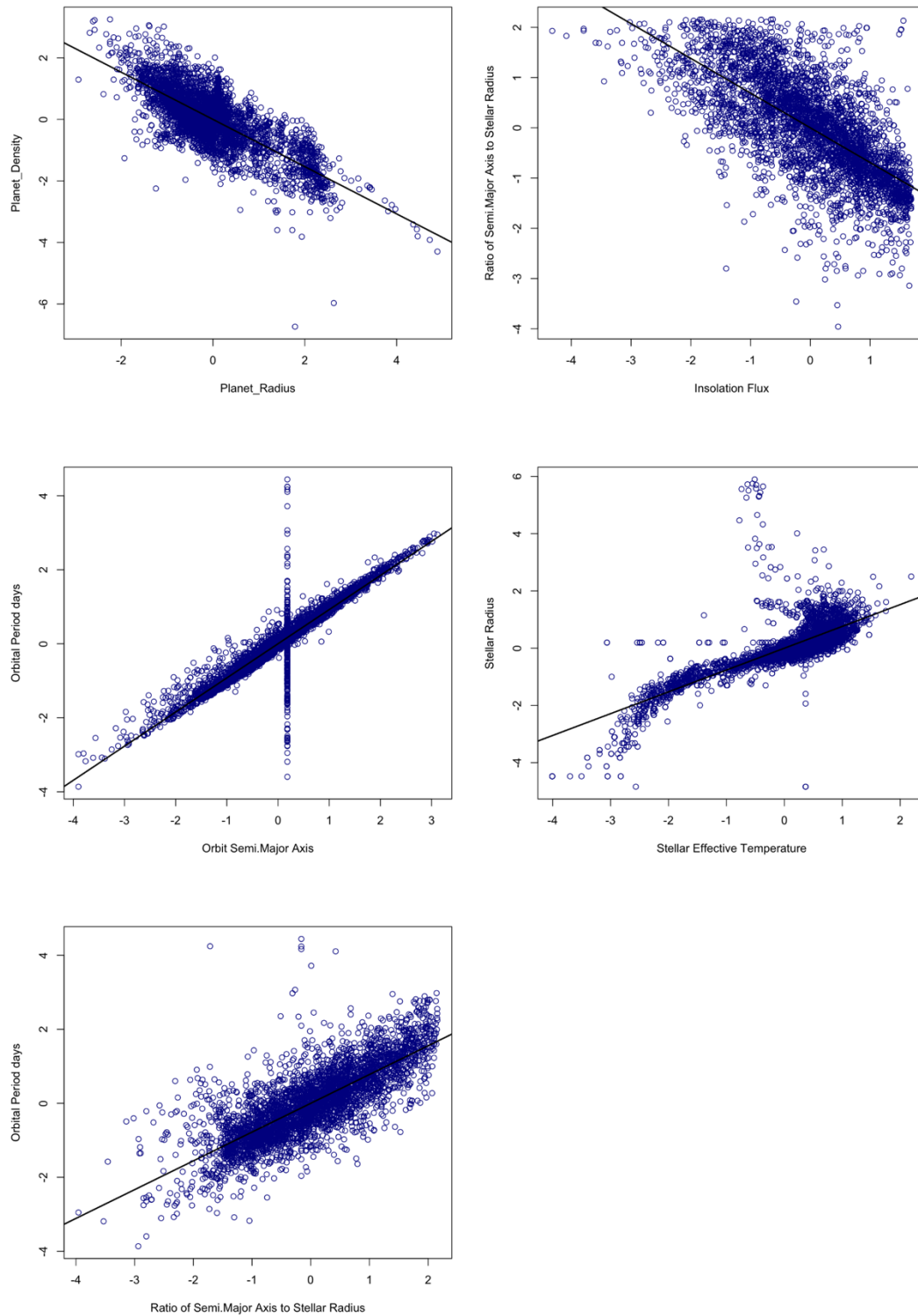


Figure 17: Scatter Plots of Variables with the Highest Correlation

Variables between 10% and 40% of null values were handled through imputation using the regression method. Imputation through regression is more statistically valid than median imputation because it preserves relationships between variables. Several variables contained over 70% of null values: Planet Mass, Planet Density, Stellar Luminosity, and Stellar Age. The Stellar Luminosity and Stellar Age variables were removed from the dataset, while the Planet Mass and Planet Density variables were retained due to their significance in planet classification. Planet Mass and Planet Density also show strong correlations with other variables, enhancing the accuracy of imputation. Once radius and mass had been fully imputed, null values for planet density could be mathematically calculated using the formula:

$$density = \frac{M}{(4/3)\pi R^3}$$

Figure 18: Formula for Density

And therefore, regression analysis was not required. During the imputation stage, for the variables Planet Mass, Planet Radius, Eccentricity, Insolation Flux, Equilibrium Temperature, and Ratio of Semi-Major Axis to Stellar Radius, the random forest package was installed in R. Predictor variables were selected from the correlation matrix figure based on their strength. The training set included the rows without missing values. The test set comprised the variable rows that needed missing values to be predicted. Regression imputation does not assume normal distribution of variables and handles non-linear relationships well by using random forest.

4.2.2 - Data Transformation

Once all null values were imputed, data transformation could be executed using Python. As the variables contained values of different measurements, the data needed to be

transformed so that all data were on the same scale. Each column was first log-transformed using the numpy library in Python to attempt to reduce the skewness in the variables with unevenly distributed data. It also attempts to compress the outliers in the data, improving the linearity of each variable to allow for more accurate classification machine learning. Log-transformation was executed before standardisation so that the skewed data was dealt with before shrinking the data. Both transformation techniques were required for this data so that the heavily tailed data was normally shaped, and to ensure all features were comparable.

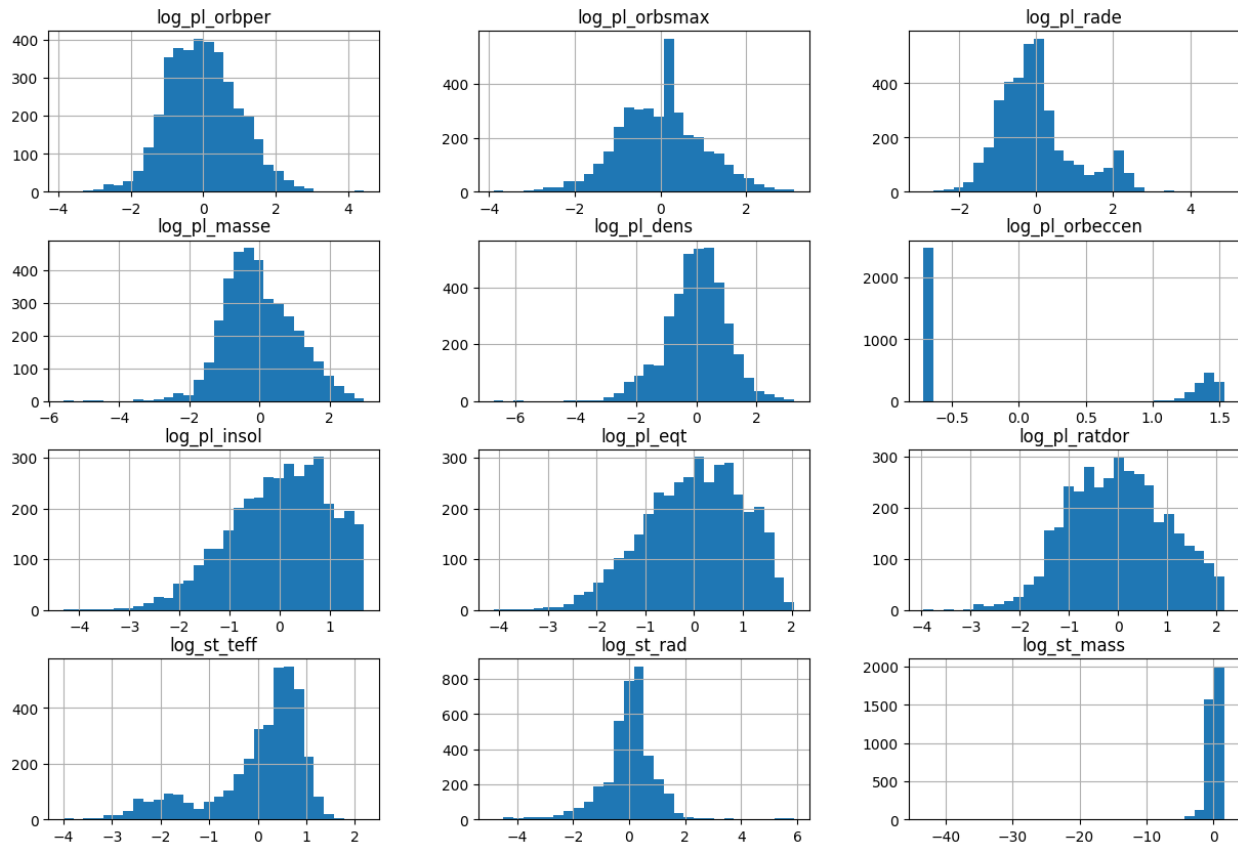


Figure 19: Histograms of the Dataset Variables after Log-Transformation

After log-transformation, it can be seen from the histograms in Figure 14 that the data which required transformation had been more evenly distributed. Features Eccentricity and Stellar Mass still showed highly skewed and unevenly distributed values after log-transformation. Log-transformation struggles with dealing with values of zero. Eccentricity is a measure of how elongated an exoplanet's orbit is, ranging from 0 (circular) to 1 (parabolic). Many rows

contained a perfect circular orbit, meaning a lot of values returned were zero in the feature, therefore making the log-transformation struggle to deal with the skewness. Based on the results from log-transformation, these two variables were dropped before analysis. Eccentricity also did not show a strong relation with the other variables based on the correlation matrix in Figure 16, suggesting it would not have had a significant role in the classification of exoplanets.

Standardisation was then executed in Python using StandardScaler from the sklearn library. Because variables in the dataset contained different measurements such as temperature (K), Density [g/cm^3] and mass, it was necessary to standardise the data so that all variables contributed equally to model training. This is important for algorithms such as KNN and SVM, which are used in this project, ensuring that certain features cannot dominate or overpower others.

4.2.3 - Response Variable

The raw dataset from the exoplanet archive did not contain a target variable required for the classification algorithms KNN, SVM and Decision Tree. The response variable was created based on the planet measurements described by NASA. Other partners, such as Kepler and TESS use these ranges, but exact thresholds vary across studies depending on the dataset and which features are available.

Table 3: Ranges of Measurements for the Creation of the Target Variable Planet Type

Planet Type	Radius (Earth radii)	Mass (Earth masses)
Terrestrial	< 1.6	< 5
Super-Earth	1.6 – 2.5	5 – 10
Neptunian	2.5 – 6	10 – 50
Gas Giant	> 6	> 50

Once the response variable was created, five types were created: Gas Giant, Neptunian, Super-Earth, Terrestrial, and unknown. Exoplanets that fell under the unknown category did not meet any of the planet type criteria and were excluded from the final analysis.

It was seen that most exoplanets were classified as Gas Giants. This class imbalance can negatively affect machine learning model performance, as algorithms may become biased toward the dominant class, making it more difficult to correctly classify underrepresented types such as Terrestrial or Super-Earth planets (see Figure 20) (He and Garcia, 2009). Therefore, the Synthetic Minority Oversampling Technique (SMOTE) was used to balance out the dataset. Training and testing data were split at a ratio of 80:20 before SMOTE was applied. After the resampling, there was a total of 1364 rows for each planet type.

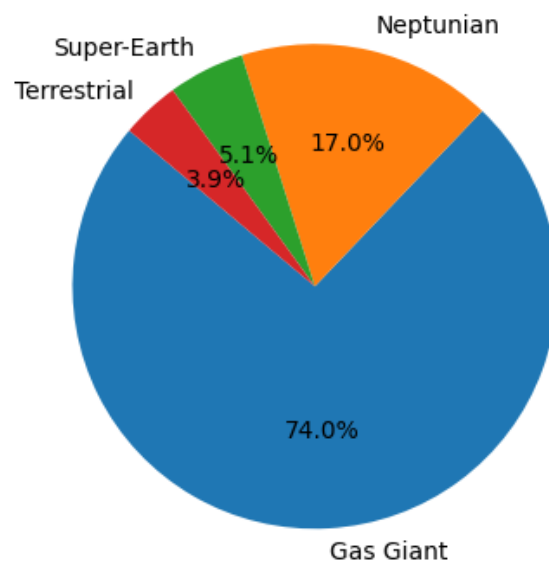


Figure 20: The Distribution of Planet Types Before SMOTE was Applied

4.3 - Modelling

4.3.1 - K-Means

K-means clustering was the first machine learning model used to analyse the dataset. The unsupervised model attempted to separate the samples in the data into n groups of equal variance. The centroids were chosen so that the sum of squares was minimised.

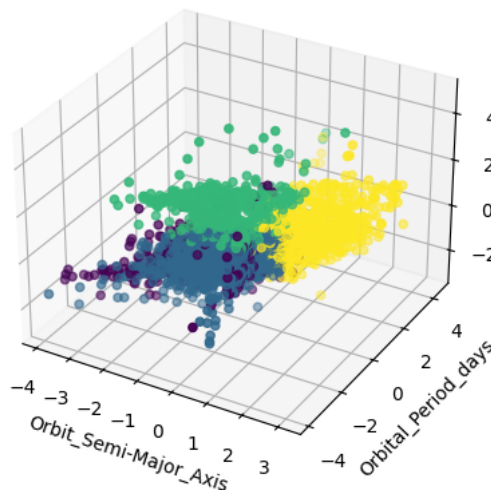


Figure 21: 3d Clustering Results Showing Four Groups

As seen in Figure 21, K-means struggled to identify and separate the data into four clear groups of exoplanets. Other numbers of groups were tried, which also did not separate the data into clear groups. The model may have struggled due to feature overlap, as many of the features were highly correlated, meaning separability would have been affected. Internal cluster validation metrics such as the WCSS may have also performed poorly, suggesting that no value of k would be strong at clustering the data.

4.3.2 - Support Vector Machine (SVM)

SVM was implicated after the imbalance of data had been evened using SMOTE. This model is a powerful predictive classification model that responds well to outliers and can be effective with complex data using a Kernel parameter. To maximise the accuracy of the algorithm, the OvO method was implemented so that all explanatory variables could be used to classify data based on the target variable. The radial basis function (RBF) Kernel function was added in this algorithm, which is the default kernel for SVM in scikit-learn. The two parameters which must be considered when implementing RBF are the gamma and C. The gamma was set to scale, which is the default Kernel coefficient and controls each individual training sample on the decision boundary. C was equal to one, allowing for a certain degree of misclassification tolerance, also known as applying regularisation. Using the Kernel function instead of other matrix transformation methods improves the model's performance.

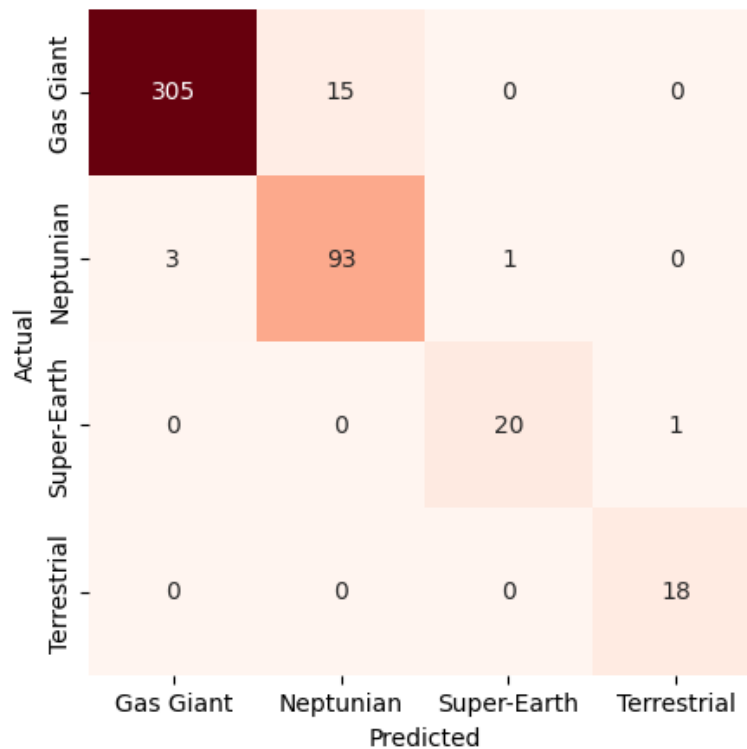


Figure 22: Confusion Matrix for SVM

The confusion matrix shows that SVM is strong at classifying the dataset, predicting 96% correctly. The model struggled the most at predicting Neptunian planets, 14% of cases predicting them as Gas Giant planets.

4.3.3 - Decision Tree

The Decision Tree model started with the root node containing all training cases in the data. It then computed its purity for the target values, which when all the nodes have purity above a threshold, it then stops (see Figure 23). The best split from the Decision Tree was the first split at the root node, as it showed the greatest reduction in Gini impurity. It best separates the data, partitioning 1,820 samples into two child nodes, one with 1,326 samples and the other with 494 samples.

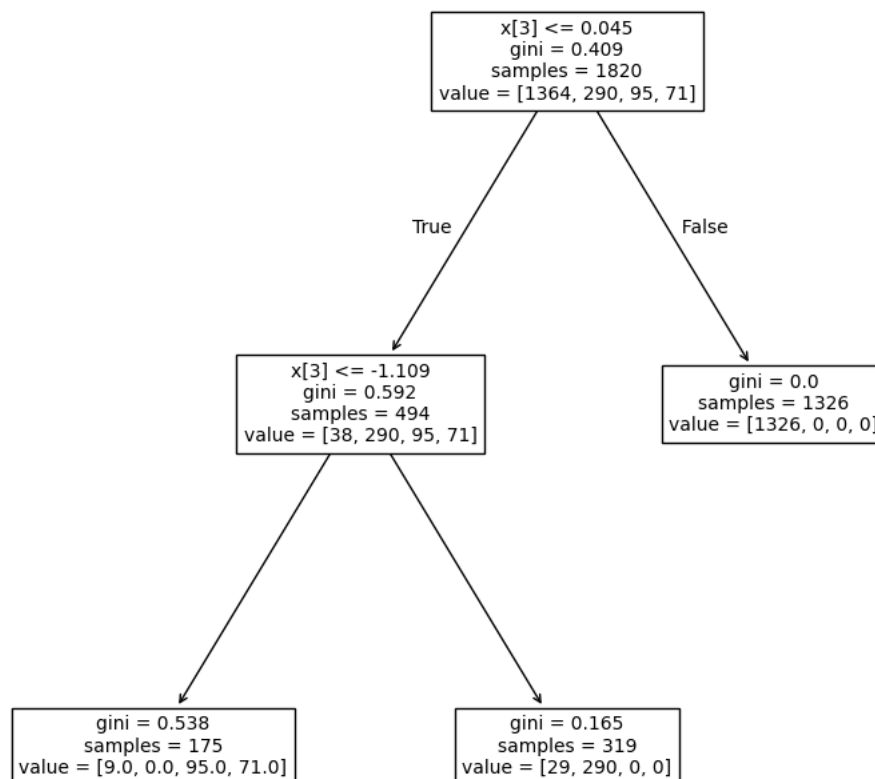


Figure 23: Decision Tree

X [3] shows that the fourth column was used in the root split, which was Planet Mass. This is in accordance with previous research that states that planet mass is a major factor in determining exoplanet types.

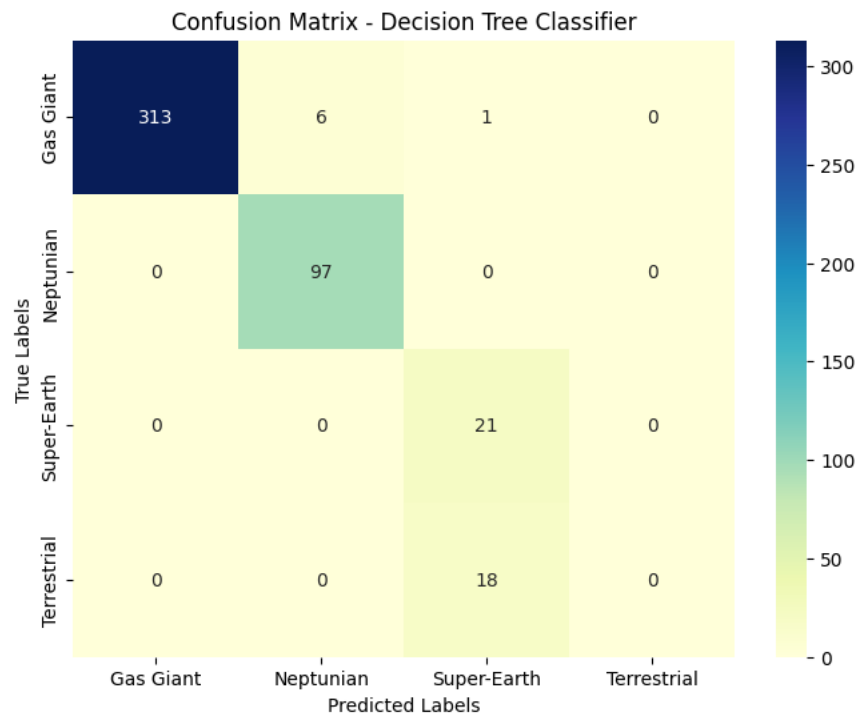


Figure 24: Confusion Matrix for the Decision Tree Model

The confusion matrix (Figure 24) shows that the model was strong at classifying Gas Giants and Neptunian planets. The model fails to identify Terrestrial exoplanets and instead categorises them as super-Earth types. This suggests the model underfitted the data.

4.3.4 - K-Nearest Neighbour (KNN)

The KNN machine learning model uses a total of five as the number of K neighbours as its default value. A value which is too small means the noise will have a high influence on the result, causing overfitting, and a large value would result in underfitting as the boundaries become too smooth. Based on the elbow method in Figure 25, the optimum number of K neighbours for the data was 3. Odd numbers of K are usually preferred in this type of model.

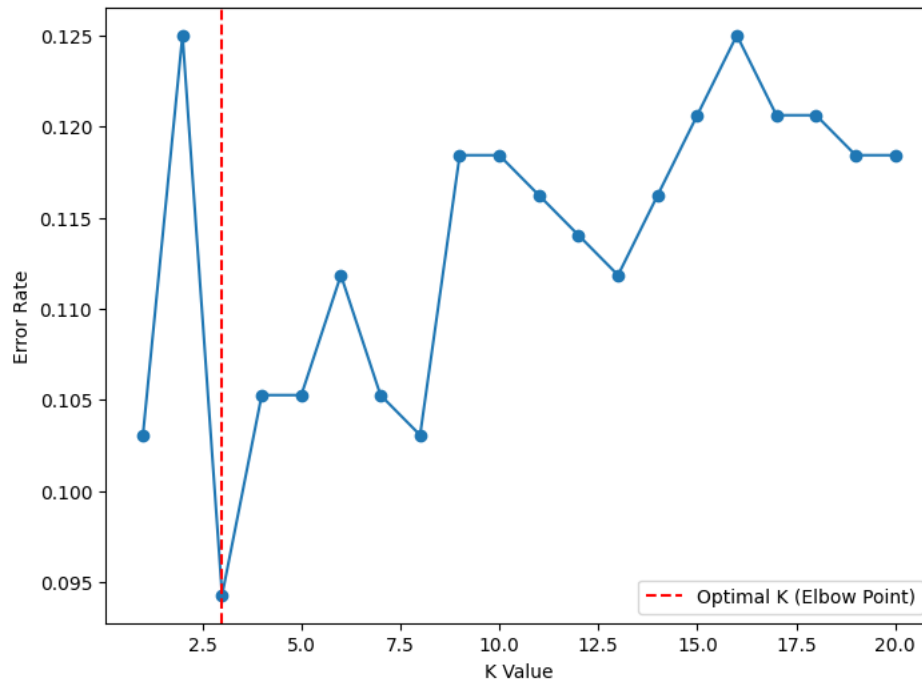


Figure 25: The Elbow Method for Finding the Optimum Value of K

After the model was implemented, the confusion matrix figure showed that KNN was able to identify all four exoplanet types. The model did classify some Gas Giant planets as Neptunian, and falsely classified Terrestrial and Neptunian planets as super-Earth planets.

The confusion matrix for KNN shows that the model could identify and classify all four exoplanet types. The model did falsely predict certain rows, predicting Gas Giants as Neptunian and Terrestrials as Super-Earths.

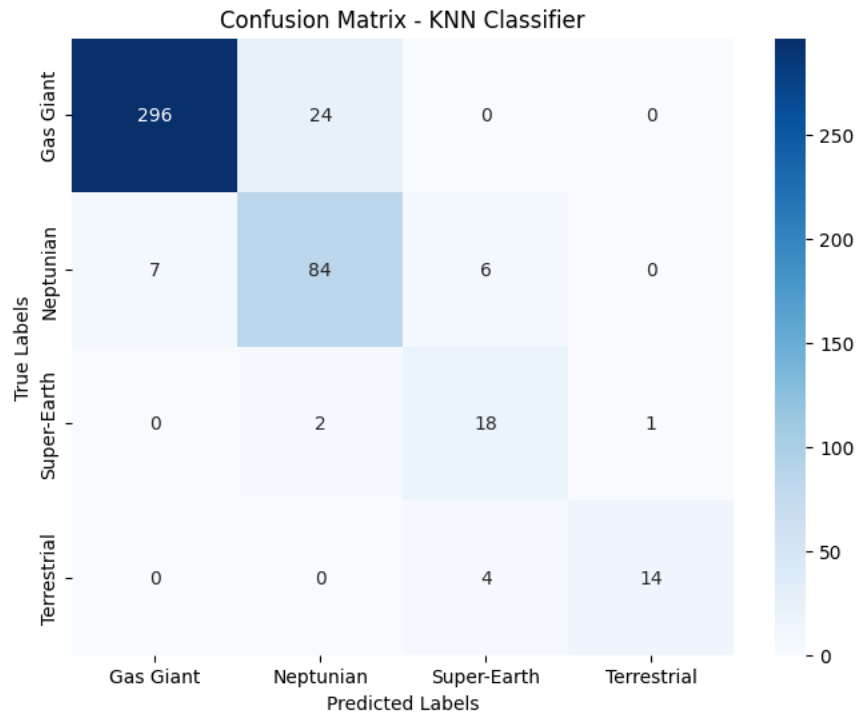


Figure 26: Confusion Matrix for the KNN Model

4.4 - Model Comparison

The performance of the machine learning models was compared using classification reports and visualised using ROC curve figures. The classification report table (see Table 4) showed that SVM had an accuracy of 0.96, Decision Tree of 0.95 and KNN of 0.9. Accuracy is the sum of true positives and true negatives divided by the total number of samples. SVM showed the best overall accuracy, with KNN being the least accurate, showing that SVM handles classes well. The model data should be balanced for the accuracy score to show reliable results. As Gas Giants make up 70% of the data, the accuracy score may be misleading therefore, other performance measures should be considered due to the data being moderately imbalanced.

Table 4: The Classification Report of the KNN, Decision Tree and SVM Model for Model Comparison

KNN					Decision Tree				
Planet Type	precision	recall	f1-score	support	Planet Type	precision	recall	f1-score	support
Gas Giant	0.98	0.93	0.95	320	Gas Giant	1	0.98	0.99	320
Neptunian	0.76	0.87	0.81	97	Neptunian	0.94	1	0.97	97
Super-Earth	0.64	0.86	0.73	21	Super-Earth	0.53	1	0.69	21
Terrestrial	0.93	0.78	0.85	18	Terrestrial	0	0	0	18
accuracy			0.9	456	accuracy			0.95	456
macro avg	0.83	0.86	0.84	456	macro avg	0.62	0.74	0.66	456
weighted avg	0.91	0.9	0.91	456	weighted avg	0.93	0.95	0.93	456

SVM				
Planet Type	precision	recall	f1-score	support
Gas Giant	0.99	0.95	0.97	320
Neptunian	0.86	0.96	0.91	97
Super-Earth	0.95	0.95	0.95	21
Terrestrial	0.95	1	0.97	18
accuracy			0.96	456
macro avg	0.94	0.97	0.95	456

Precision and recall were also used to compare model performance. Precision calculates how often a prediction is actually true. The highest precision for Gas Giant exoplanets was from the Decision Tree model with a score of 100%. The lowest was KNN with a score of 98%. This suggests that all models successfully and consistently predicted this exoplanet type. The highest Neptunian precision score was 94% from the Decision Tree model, and the lowest was from the KNN model at 76%. This is a greater difference in correct predictions, in contrast to predicting Gas Giants, which could be down to the imbalance in data. For Super-Earth type planets, the highest precision score was from the SVM model at 95%, with the lowest score coming from the KNN model at 64%. For Terrestrial exoplanets, the highest precision score was 95% by the SVM model, and the lowest was the Decision Tree model, which failed to identify any of these types of planets. Based on the precision score values, SVM remained the most successful algorithm at predicting exoplanet types.

The macro average score averages all classes equally. SVM is significantly greater in this performance measure as it resulted in a score of 94% compared to 83% and 62% for the KNN and Decision Tree models, respectively. The result from the Decision Tree model suggests that it underperforms on specific smaller cases, which is in line with the precision score of

the Terrestrial and Super-Earth type exoplanets. Due to the slight imbalance in the dataset, the macro average values and the F1-score are more reliable indicators of model performance in comparison to the accuracy scores.

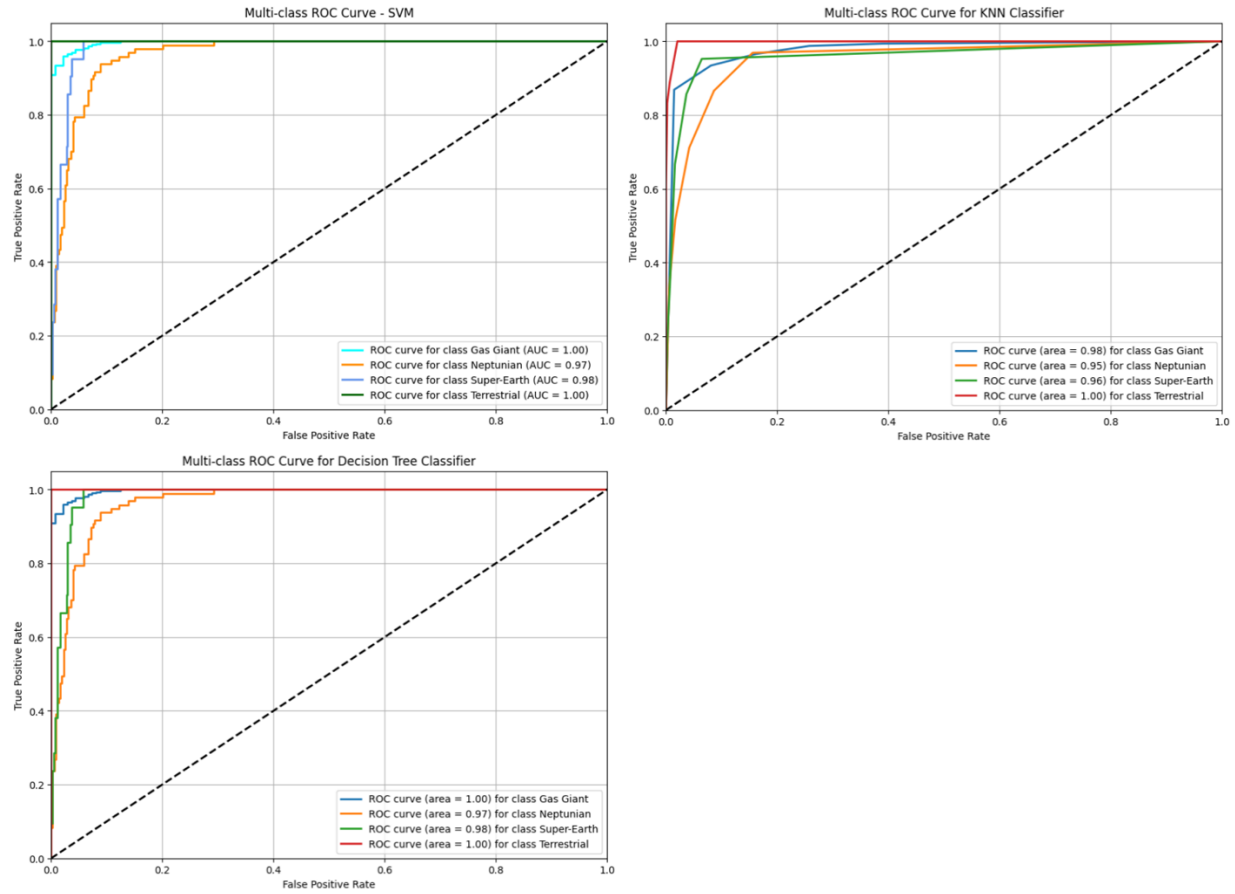


Figure 27: ROC Curve Visualisation of the SVM, KNN and Decision Tree Models Performance

The ROC curve figures show the strength of each machine learning model for the four exoplanet types in the target variable. This visually shows the classification report, showing the rate of true positive predictions.

5 - Conclusion

5.1 - Conclusion

Machine learning was carried out in the project with the aim of correctly classifying exoplanets. As new techniques and missions have allowed for an increase in the detection rate of exoplanets, machine learning techniques can be used as an efficient way of analysing the data. Classifying is one way in which exoplanets can be analysed to produce useful information and understand more about the data. As the dataset contained a significant number of missing values, an effective methodology and process were required using the CRISP-DM model to ensure deadlines were met and all the important steps were executed.

To build an accurate model, data exploration and preparation were executed. It was ensured that missing values were identified and correctly dealt with using efficient techniques. Columns were either dropped or the null values were imputed using various techniques, depending on the percentage of missing values. This stage was crucial in creating an accurate dataset for analysis. It must be considered in this project that a lot of the data used for analysis was imputed and was not official metrics of each of the discovered exoplanets.

Unsupervised machine learning using K-means clustering struggled to group the data without predefined labels, however, changes in its parameters or other unsupervised techniques may have performed better. A target variable was manually created as none were present in the raw data. Among the three supervised models used throughout this project, SVM exhibited the best performance in accurately classifying exoplanets, identifying all four types. Previous research has managed to successfully cluster exoplanet data with a seven-dimensional parameter space using an algorithm on 10,000 simulated planets. Six clusters were identified by eye (Lalande *et al.*, 2024). This suggests that unsupervised methods can be used as they were able to identify planet types such as Gas Giants. The most informative features which influenced the clusters were eccentricity, radius, orbital periods and planet

mass. These align with the features used in this project, in which the correlation matrix showed high relationships between them. It is preferred to use a complete dataset of exoplanets, and imputation in previous research had also shown a reduction in error rate compared to a 150-planet test set. This backs the use of imputation instead of removing all rows containing null values (Lalande *et al.*, 2024).

5.2 - Recommendation for Future Work

This project successfully built effective classification machine learning algorithms to classify exoplanets. Future research could investigate whether other unsupervised techniques can successfully cluster exoplanets and compare the results to supervised techniques. Using a clustering algorithm without predefined labels may suggest new ways of grouping exoplanets compared to its current way, with discoveries of other exoplanets happening more frequently than before. Also, different imputation techniques can be revised for this dataset, as not all features can have their actual measurements present in the dataset due to various discovery methods used. Finding the imputation measurement with the greatest accuracy is crucial as it will allow for the full dataset of exoplanets to be used in machine learning and data analysis. Other machine techniques other than supervised or unsupervised may be of use for exoplanet analysis, such as reinforcement learning.

References

Alpaydin, E. (2014) Introduction to Machine Learning. 3rd edn. Cambridge, MA: The MIT Press.

Charbonneau, D., Brown, T.M., Latham, D.W. et al. (2000) 'Detection of Planetary Transits Across a Sun-like Star', *Astrophysical Journal Letters*, 529, pp. L45–L48.

Chatterjee, A., Panja, S. and Yasmin, G. (2018) 'Kernel Functions of SVM: A Comparison and Optimal Solution', *Proceedings of the Second International Conference ICAICR 2018, Shimla, India, July 14–15, 2018*. Available at: https://www.researchgate.net/publication/329606253_Kernel_Functions_of_SVM_A_Comparison_and_Optimal_Solution_Second_International_Conference_ICAICR_2018_Shimla_India_July_14-15_2018_Revised_Selected_Papers_Part_I

Choi, Y.J., Nam, C.M. and Kwak, M.J. (2004) 'Multiple imputation technique applied to appropriateness ratings in cataract surgery', *Yonsei Medical Journal*, 45(5), pp. 829–837.

Deeg, H.J. and Alonso, R. (2018) 'Transit photometry as an exoplanet discovery method', *Handbook of Exoplanets*, pp. 1–24. Available at: <https://arxiv.org/abs/1803.07867> (Accessed: 19 February 2025).

Deisenroth, M.P., Faisal, A.A. and Ong, C.S. (2020) *Mathematics for Machine Learning*. Cambridge: Cambridge University Press.

Fotache, M. and Strimbei, C. (2015) 'SQL and data analysis. Some implications for data analysts and higher education', *Procedia Economics and Finance*, 20, pp. 243–251.

Gaudi, B.S. (2012) ‘Microlensing Surveys for Exoplanets’, *Annual Review of Astronomy and Astrophysics*, 50, pp. 411–453. doi:10.1146/annurev-astro-081811-125518

Gholamy, A., Kreinovich, V. and Kosheleva, O. (2018) ‘Why 70/30 or 80/20 relation between training and testing sets: A pedagogical explanation’, *International Journal of Intelligent Technologies and Applied Statistics*, 11(2), pp. 105–111.

Han, J., Kamber, M. and Pei, J. (2011) *Data Mining: Concepts and Techniques*. 3rd edn. Morgan Kaufmann.

Hayes, J.J.C. et al. (2020) ‘Optimizing exoplanet atmosphere retrieval using unsupervised machine-learning classification’, *Monthly Notices of the Royal Astronomical Society*, 494(4), pp. 4492–4508. doi:10.1093/mnras/staa978.

He, H. and Garcia, E.A. (2009) ‘Learning from imbalanced data’, *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263–1284. doi:10.1109/TKDE.2008.239

Huilgol, P. (2024) ‘Precision and recall in machine learning’, *Analytics Vidhya*. Available at: <https://www.analyticsvidhya.com/articles/precision-and-recall-in-machine-learning.com>

Ian H. Witten, Frank, E., Hall, M.A. and Pal, C.J. (2016) *Data Mining: Practical Machine Learning Tools and Techniques*. Elsevier Science.

Jia, Z. and Gang, Z. (2012) ‘A Statistical Survey of Orbital Parameters of Extra-Solar Planets System’, *Progress in Astronomy*, 30, pp. 48–63.

Jijo, B.T. and Abdulazeez, A.M. (2021) ‘Classification based on decision tree algorithm for machine learning’, *Journal of Applied Science and Technology Trends*, 2(1), pp. 20–28.

Jin, X. and Han, J. (2011) 'K-Means Clustering', in Sammut, C. and Webb, G.I. (eds.) Encyclopedia of Machine Learning. Boston, MA: Springer. https://doi.org/10.1007/978-0-387-30164-8_425

Kaelbling, L.P., Littman, M.L. and Moore, A.W. (1996) 'Reinforcement learning: a survey', Journal of Artificial Intelligence Research, 4, pp. 237–285.

Kataria, G. and Singh, K. (2013) 'A review of data classification using K-Nearest Neighbour algorithm', International Journal of Emerging Technology and Advanced Engineering, 3(6), pp. 354–360.

Khan, M.A. et al. (2024) 'A Review of Exoplanet Detection Processes using Artificial Intelligence Techniques', International Journal of Computer Applications, 186(36), pp. 1–6. Available at: <https://www.ijcaonline.org/archives/volume186/number36/khan-2024-ijca-923940.pdf>

Krebs, R. et al. (2024) 'Applying multi-class support vector machines: One-vs-One vs. One-vs-All on the UWF-ZeekDataFall22 dataset', Electronics, 13, p. 3916. doi:10.3390/electronics13193916.

Lawlor, K.B. and Hornyak, M.J. (2012) 'Smart Goals: How the Application of Smart Goals can Contribute to Achievement of Student Learning Outcomes', Developments in Business Simulation and Experiential Learning, 39, pp. 259–267.

Lindholm, A. et al. (2022) Machine Learning: A First Course for Engineers and Scientists. Cambridge University Press. doi:10.1017/9781108919371.

Mahesh, B. (2018) 'Machine Learning Algorithms – A Review', International Journal of Science and Research (IJSR), 7(6), pp. 381–386. Available at: <https://www.ijsr.net/archive/v7i6/ART20182328.pdf>

Malik, A., Moster, B.P. and Obermeier, C. (2022) 'Exoplanet detection using machine learning', Monthly Notices of the Royal Astronomical Society, 513(4), pp. 5505–5516. <https://doi.org/10.1093/mnras/stab3692>

Marutho, D. et al. (2018) 'The determination of cluster number at K-Mean using elbow method and purity evaluation on headline news', 2018 International Seminar on Application for Technology of Information and Communication (iSemantic), pp. 221–225. doi:10.1109/ISEMANTIC.2018.8549751

McKinney, W. (2022) Python for Data Analysis. Sebastopol, CA: O'Reilly Media. Available at: <https://search.ebscohost.com/login.aspx?direct=true&AuthType=ip,sso&db=nlebk&AN=3360238&site=ehost-live> (Accessed: 7 April 2025).

Metre, K.V. et al. (2024) 'An Introduction to Power BI for Data Analysis', International Journal of Intelligent Systems and Applications in Engineering, 12(1s), pp. 142–147.

Mohammed, J.Z. and Meira, W. Jr. (2020) Data Mining and Machine Learning: Fundamental Concepts and Algorithms. 2nd ed. Cambridge University Press.

Morrow, J. (2013) 'Brief Introduction to the 12 Steps of Evaluation Data Cleaning', SlideShare. Available at: <https://www.slideshare.net/slideshow/brief-introduction-to-the-12-steps-of-evaluagio/26168236> (Accessed: 2 May 2025).

NASA (n.d.) Exoplanet planet types, NASA Science. Available at: <https://science.nasa.gov/exoplanets/planet-types/> (Accessed: 17 March 2025).

NASA Exoplanet Science Institute (NExSci). (2024) NASA Exoplanet Archive. California Institute of Technology. Available at: <https://exoplanetarchive.ipac.caltech.edu/>

Rokach, L. and Maimon, O. (2008) Data mining with decision trees: Theory and applications. World Scientific Publishing Company.

Roth, P.L. (1994) 'Missing data: A conceptual review for applied psychologists', *Personnel Psychology*, 47(3), pp. 537–560.

Rubin, L.H. et al. (2007) 'Methods for handling missing data in the behavioral neurosciences: Don't throw the baby rat out with the bath water', *Journal of Undergraduate Neuroscience Education*, 5(2), p. A71.

Saini, A. (2024) 'Guide on Support Vector Machine (SVM) Algorithm', *Analytics Vidhya*. Available at: <https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/>

Sarker, I.H. (2021) 'Machine learning: Algorithms, real-world applications and research directions', *SN Computer Science*, 2, p. 160. doi:10.1007/s42979-021-00592-x

Schröer, C., Kruse, F. and Gómez, J.M. (2021) 'A systematic literature review on applying CRISP-DM process model', *Procedia Computer Science*, 181, pp. 526–534.

Silver, D. et al. (2016) 'Mastering the game of Go with deep neural networks and tree search', *Nature*, 529(7587), pp. 484–489. doi:10.1038/nature16961.

Smith, M.J. (2022) 'Using deep learning to explore ultra-large scale astronomical datasets', MALCOM: Indonesian Journal of Machine Learning and Computer Science, 4(3), pp. 753–763.

Sturrock, G.C., Manry, B. and Rafiqi, S. (2019) 'Machine learning pipeline for exoplanet classification', SMU Data Science Review, 2(1), Article 9. Available at: <https://scholar.smu.edu/datasciencereview/vol2/iss1/9>

Taiwo, O.A. (2010) 'Types of machine learning algorithms', in Zhang, Y. (ed.) New Advances in Machine Learning. University of Portsmouth, United Kingdom: InTech, pp. 3–31. Available at: <http://www.intechopen.com/books/new-advances-inmachine-learning/types-of-machine-learning-algorithms>

Tsapras, Y. (2018) 'Microlensing Searches for Exoplanets', Geosciences, 8(10), p. 365. doi:10.3390/geosciences8100365.

Valizadegan, H. et al. (2022) 'ExoMiner: a highly accurate and explainable deep learning classifier that validates 301 new exoplanets', The Astrophysical Journal, 926(2), p. 120.

Wright, J.T. (2013) 'Exoplanet Detection Methods', in Oswalt, T.D., French, L.M. and Kalas, P. (eds.) Planets, Stars and Stellar Systems. Dordrecht: Springer, pp. 489–540. Available at: <https://arxiv.org/abs/1707.07983> (Accessed: 28 February 2025).

Zaki, M.J. and Meira, W. Jr. (2021) 'Machine Learning: Algorithms, Real-World Applications, and Research Directions', SN Computer Science, 2(5), pp. 1–15. Available at: <https://link.springer.com/article/10.1007/s42979-021-00592-x>

Appendix

```
1 ## Installing packages ##
2
3 install.packages("readr")
4 library(readr)
5 install.packages('dplyr')
6 library(dplyr)
7 install.packages("tidyverse")
8 library('tidyverse')
9 install.packages("ggplot2")
10 library("ggplot2")
11 install.packages("corrplot")
12 library("corrplot")
13
14 ## Data Importing ##
15 exoplanet_dataset <- read.csv("/Users/ashan/Desktop/PostGrad - Data Analytics/Dissertation/D
16                               na.strings = c("", "NA"))
17
18 ## Data Exploration ##
19 sum(is.na(exoplanet_dataset$st_mass))
20 head(exoplanet_dataset)
21 names(exoplanet_dataset)
22 sum(is.na(exoplanet_dataset))
23
24 nrow(exoplanet_dataset)
25 ncol(exoplanet_dataset)
26 122*38063
27 sum(is.na(exoplanet_dataset))
28 (1379966/4643686)*100
29 ## 29.7% of missing values ##
30
31 sum(duplicated(exoplanet_dataset))
32
33 exoplanet_dataset_new <- c("pl_name", "pl_orbper", "pl_orbsmax", "pl_rade", "pl_masse",
34                             "pl_dens", "pl_orbeccen", "pl_insol", "pl_eqt", "pl_ratdor",
35                             "st_teff", "st_rad", "st_mass", "st_metratio", "st_lum", "st_age"
36                             "st_dens")
37
38
39 exoplanet_dataset_cleaned <- new_exoplanet_dataset %>%
40   arrange(rowSums(is.na(select(., all_of(exoplanet_dataset_new))))) %>%
41   distinct(pl_name, .keep_all = TRUE)
42
43 exoplanet_dataset %>% filter(pl_name == "14 Her b")
44 nrow(exoplanet_dataset %>% filter(pl_name == "14 Her b"))
45 sum(is.na(exoplanet_dataset %>% filter(pl_name == "14 Her b")))
46 sum(is.na(exoplanet_dataset_cleaned %>% filter(pl_name == "14 Her b")))
47 nrow(exoplanet_dataset_cleaned %>% filter(pl_name == "14 Her b"))
48
49 exoplanet_dataset_cleaned
50 head(exoplanet_dataset_cleaned)
51 ncol(exoplanet_dataset_cleaned)
52 nrow(exoplanet_dataset_cleaned)
53 sum(duplicated(exoplanet_dataset_cleaned))
54 sum(is.na(exoplanet_dataset_cleaned))
55
56 null_values <- sum(is.na(exoplanet_dataset_cleaned))
57 datapoints <- 5819 * 17
58 datapoints
59 null_values
60 Null_values_percentage <- (null_values/datapoints)*100
61 Null_values_percentage
62 ## 30.4% of missing values in the cleaned data ##
63
64 ?hist
65
66 hist(exoplanet_dataset_cleaned$pl_masse, main = paste(), xlab = "Planet Mass [Earth Mass]")
67 hist(exoplanet_dataset_cleaned$pl_rade, main = paste(), xlab = "Planet Radius [Earth Radius]")
68 hist(exoplanet_dataset_cleaned$pl_eqt, main = paste(), xlab = "Equilibrium Temperature [K]")
69 hist(exoplanet_dataset_cleaned$st_mass, main = paste(), xlab = "Stellar Mass [Solar mass]")
70 hist(exoplanet_dataset_cleaned$pl_orbper, main = paste(), xlab = "Orbital Period [days]")
71 hist(exoplanet_dataset_cleaned$pl_orbsmax, main = paste(), xlab = "Orbit Semi-Major Axis [a]")
72 hist(exoplanet_dataset_cleaned$pl_dens, main = paste(), xlab = "Planet Density [g/cm**3]")
73 hist(exoplanet_dataset_cleaned$pl_orbeccen, main = paste(), xlab = "Eccentricity")
74 hist(exoplanet_dataset_cleaned$pl_insol, main = paste(), xlab = "Insolation Flux [Earth Flux]")
```

```

74 hist(exoplanet_dataset_cleaned$pl_ratdor, main = paste(), xlab = "Ratio of Semi-Major Axis to")
75 hist(exoplanet_dataset_cleaned$st_teff, main = paste(), xlab = "Stellar Effective Temperature")
76 hist(exoplanet_dataset_cleaned$st_rad, main = paste(), xlab = "Stellar Radius [Solar Radius]")
77 hist(exoplanet_dataset_cleaned$st_lum, main = paste(), xlab = "Stellar Luminosity [log(Solar)]")
78 hist(exoplanet_dataset_cleaned$st_age, main = paste(), xlab = "Stellar Age [Gyr]")
79 hist(exoplanet_dataset_cleaned$st_dens, main = paste(), xlab = "Stellar Density [g/cm**3]")
80
81 boxplot(exoplanet_dataset_cleaned$pl_masse, main = paste(), xlab = "Planet Mass [Earth Mass]")
82 boxplot(exoplanet_dataset_cleaned$pl_rade, main = paste(), xlab = "Planet Radius [Earth Radius]")
83 boxplot(exoplanet_dataset_cleaned$pl_eqt, main = paste(), xlab = "Equilibrium Temperature [K]")
84 boxplot(exoplanet_dataset_cleaned$st_mass, main = paste(), xlab = "Stellar Mass [Solar mass]")
85 boxplot(exoplanet_dataset_cleaned$pl_orbper, main = paste(), xlab = "Orbital Period [days]")
86 boxplot(exoplanet_dataset_cleaned$pl_orbsmax, main = paste(), xlab = "Orbit Semi-Major Axis [AU]")
87 boxplot(exoplanet_dataset_cleaned$pl_dens, main = paste(), xlab = "Planet Density [g/cm**3]")
88 boxplot(exoplanet_dataset_cleaned$pl_orbeccen, main = paste(), xlab = "Eccentricity")
89 boxplot(exoplanet_dataset_cleaned$pl_insol, main = paste(), xlab = "Insolation Flux [Earth Flux]")
90 boxplot(exoplanet_dataset_cleaned$pl_ratdor, main = paste(), xlab = "Ratio of Semi-Major Axis to")
91 boxplot(exoplanet_dataset_cleaned$st_teff, main = paste(), xlab = "Stellar Effective Temperature")
92 boxplot(exoplanet_dataset_cleaned$st_rad, main = paste(), xlab = "Stellar Radius [Solar Radius]")
93 boxplot(exoplanet_dataset_cleaned$st_lum, main = paste(), xlab = "Stellar Luminosity [log(Solar)]")
94 boxplot(exoplanet_dataset_cleaned$st_age, main = paste(), xlab = "Stellar Age [Gyr]")
95 boxplot(exoplanet_dataset_cleaned$st_dens, main = paste(), xlab = "Stellar Density [g/cm**3]")
96
97 sum(apply(exoplanet_dataset_cleaned, 1, function(x) any(is.na(x))))
98 sum(is.na(exoplanet_dataset_cleaned$pl_masse))
99 sum(is.na(exoplanet_dataset_cleaned$pl_orbper))
100 sum(is.na(exoplanet_dataset_cleaned$pl_orbsmax))
101 sum(is.na(exoplanet_dataset_cleaned$pl_rade))
102 sum(is.na(exoplanet_dataset_cleaned$pl_dens))
103 sum(is.na(exoplanet_dataset_cleaned$pl_orbeccen))
104 sum(is.na(exoplanet_dataset_cleaned$pl_insol))
105 sum(is.na(exoplanet_dataset_cleaned$pl_eqt))
106 sum(is.na(exoplanet_dataset_cleaned$pl_ratdor))
107 sum(is.na(exoplanet_dataset_cleaned$st_teff))
108 sum(is.na(exoplanet_dataset_cleaned$st_rad))
109 sum(is.na(exoplanet_dataset_cleaned$st_mass))

115 summary(exoplanet_dataset_cleaned$pl_masse)
116 summary(exoplanet_dataset_cleaned$pl_orbper)
117 summary(exoplanet_dataset_cleaned$pl_orbsmax)
118 summary(exoplanet_dataset_cleaned$pl_rade)
119 summary(exoplanet_dataset_cleaned$pl_dens)
120 summary(exoplanet_dataset_cleaned$pl_orbeccen)
121 summary(exoplanet_dataset_cleaned$pl_insol)
122 summary(exoplanet_dataset_cleaned$pl_eqt)
123 summary(exoplanet_dataset_cleaned$pl_ratdor)
124 summary(exoplanet_dataset_cleaned$st_teff)
125 summary(exoplanet_dataset_cleaned$st_rad)
126 summary(exoplanet_dataset_cleaned$st_mass)
127 summary(exoplanet_dataset_cleaned$st_lum)
128 summary(exoplanet_dataset_cleaned$st_age)
129 summary(exoplanet_dataset_cleaned$st_dens)
130
131 sum(apply(exoplanet_dataset_cleaned, 1, function(x) any(is.na(x))))
132
133 nrow(exoplanet_dataset_cleaned)
134
135 head(exoplanet_dataset_cleaned$st_metratio)
136
137 drops <- c("st_metratio", "pl_name")
138 exoplanet_dataset_cleaned2 <- exoplanet_dataset_cleaned[, !(names(exoplanet_dataset_cleaned) %in% drops)]
139
140 ncol(exoplanet_dataset_cleaned2)
141 names(exoplanet_dataset_cleaned2)
142
143 cleaned_data <- na.omit(exoplanet_dataset_cleaned2)
144 nrow(cleaned_data)
145
146 correlation_matrix <- cor(cleaned_data)
147 corplot(correlation_matrix, method = "color", type = "lower",
148         diag = FALSE, tl.cex = 0.8, tl.col = "black", cl.cex = 0.8,
149         mar = c(2,2,2,2), addCoef.col = "green", number.cex = 0.8)
150

```

```

151 cor_matrix <- cor(exoplanet_dataset_cleaned2, use = "complete.obs")
152 corrplot(cor_matrix, diag = FALSE, tl.col = "black", method = "color", type = "lower", tl.cex = 1.2)
153
154
155 write.csv(exoplanet_dataset_cleaned, "exoplanet_cleaned.csv", row.names = FALSE)
156
157
158
159 ## Handling of missing data ##
160
161 ## Low Missing (>10%) ##
162
163 cleaned_exoplanet_dataset <- read.csv("/Users/ashan/Desktop/PostGrad - Data Analytics/Dissertation/exoplanet_data.csv",
164   na.strings = c("", "NA"))
165
166 tail(cleaned_exoplanet_dataset)
167   ## Stellar Mass [Solar mass] ##
168 cleaned_exoplanet_dataset$st_mass[is.na(cleaned_exoplanet_dataset$st_mass)] <-
169   median(cleaned_exoplanet_dataset$st_mass, na.rm = TRUE)
170   ## Stellar Effective Temperature [K] ##
171 cleaned_exoplanet_dataset$st_teff[is.na(cleaned_exoplanet_dataset$st_teff)] <-
172   median(cleaned_exoplanet_dataset$st_teff, na.rm = TRUE)
173   ## Stellar Radius [Solar Radius] ##
174 cleaned_exoplanet_dataset$st_rad[is.na(cleaned_exoplanet_dataset$st_rad)] <-
175   median(cleaned_exoplanet_dataset$st_rad, na.rm = TRUE)
176   ## Orbital Period [days] ##
177 cleaned_exoplanet_dataset$pl_orbper[is.na(cleaned_exoplanet_dataset$pl_orbper)] <-
178   median(cleaned_exoplanet_dataset$pl_orbper, na.rm = TRUE)
179   ## Orbit Semi-Major Axis [au] ##
180 cleaned_exoplanet_dataset$pl_orbsmax[is.na(cleaned_exoplanet_dataset$pl_orbsmax)] <-
181   median(cleaned_exoplanet_dataset$pl_orbsmax, na.rm = TRUE)
182
183 sum(is.na(cleaned_exoplanet_dataset$st_mass))
184 sum(is.na(cleaned_exoplanet_dataset$st_teff))
185 sum(is.na(cleaned_exoplanet_dataset$st_rad))
186 sum(is.na(cleaned_exoplanet_dataset$pl_orbper))

```

```

189   ## Regression (40-80% Missing) ##
190
191 install.packages("randomForest")
192 library(randomForest)
193
194   ## 1 ## pl_rade ##
195
196 predictors_rade <- c("pl_orbper", "st_mass", "st_rad", "st_teff")
197
198 train_rade <- cleaned_exoplanet_dataset[!is.na(cleaned_exoplanet_dataset$pl_rade), ]
199 test_rade <- cleaned_exoplanet_dataset[is.na(cleaned_exoplanet_dataset$pl_rade), ]
200
201 test_rade_indexes <- which(is.na(cleaned_exoplanet_dataset$pl_rade))
202 test_rade_complete <- test_rade[complete.cases(test_rade[, predictors_rade]), ]
203
204 valid_rows <- test_rade_indexes[complete.cases(test_rade[, predictors_rade])]
205
206 rf_rade <- randomForest(pl_rade ~ pl_orbper + st_mass + st_rad + st_teff,
207   data = train_rade[complete.cases(train_rade[, predictors_rade]), ])
208
209 predicted_rade <- predict(rf_rade, newdata = test_rade_complete)
210
211 cleaned_exoplanet_dataset$pl_rade[valid_rows] <- predicted_rade
212
213   ## Check ##
214 sum(is.na(cleaned_exoplanet_dataset$pl_rade))
215 summary(cleaned_exoplanet_dataset$pl_rade)
216 nrow(cleaned_exoplanet_dataset$pl_rade)
217 hist(cleaned_exoplanet_dataset$pl_rade)
218 tail(cleaned_exoplanet_dataset$pl_rade)
219 cleaned_exoplanet_dataset[cleaned_exoplanet_dataset$pl_rade > 100, c("pl_rade")]
220 cleaned_exoplanet_dataset <- cleaned_exoplanet_dataset[cleaned_exoplanet_dataset$pl_rade <= 100, ]
221 nrow(cleaned_exoplanet_dataset)
222 hist(cleaned_exoplanet_dataset$pl_rade)
223 length(cleaned_exoplanet_dataset$pl_rade)
224

```

```

402                                     ## pl_dens ##
403
404 earth_mass_grams <- 5.972e27
405 earth_radius_cm <- 6.371e8
406
407 missing_density_indexes <- which(is.na(cleaned_exoplanet_dataset$pl_dens) &
408                                !is.na(cleaned_exoplanet_dataset$pl_masse) &
409                                !is.na(cleaned_exoplanet_dataset$pl_rade))
410
411 calculated_density <- (cleaned_exoplanet_dataset$pl_masse[missing_density_indexes] * earth_m
412 ((4/3) * pi * (cleaned_exoplanet_dataset$pl_rade[missing_density_indexes] * earth_radius_cm
413
414 calculated_density_gcm3 <- calculated_density
415 cleaned_exoplanet_dataset$pl_dens[missing_density_indexes] <- calculated_density_gcm3
416
417 ## clean ##
418
419 cleaned_exoplanet_dataset$pl_dens_imputed <- FALSE
420 cleaned_exoplanet_dataset$pl_dens_imputed[missing_density_indexes] <- TRUE
421
422 table(cleaned_exoplanet_dataset$pl_dens_imputed)
423
424
425 write.csv(cleaned_exoplanet_dataset, "Imputed_exoplanet_Dataset.csv", row.names = FALSE)
426
427 nrow(cleaned_exoplanet_dataset)
428
429 hist(cleaned_exoplanet_dataset$pl_orbper)
430 hist(cleaned_exoplanet_dataset$pl_orbsmax)
431 hist(cleaned_exoplanet_dataset$pl_rade)
432 hist(cleaned_exoplanet_dataset$pl_masse)
433 hist(cleaned_exoplanet_dataset$pl_dens)
434 hist(cleaned_exoplanet_dataset$pl_orbeccen)
435 hist(cleaned_exoplanet_dataset$pl_insol)
436 hist(cleaned_exoplanet_dataset$pl_eqt)
437 hist(cleaned_exoplanet_dataset$pl_ratdor)

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("/content/Imputed_exoplanet_Dataset.csv")

df.columns

Show hidden output

columns_to_transform = ['pl_orbper', 'pl_orbsmax', 'pl_rade', 'pl_masse', 'pl_dens',
                        'pl_orbeccen', 'pl_insol', 'pl_eqt', 'pl_ratdor', 'st_teff',
                        'st_rad', 'st_mass']
for column in columns_to_transform:
    new_column_name = 'log_' + column

    df[new_column_name] = np.log(df[column])

Show hidden output

df.columns

Show hidden output

selected_columns = ['log_pl_orbper', 'log_pl_orbsmax',
                    'log_pl_rade', 'log_pl_masse',
                    'log_pl_dens', 'log_pl_orbeccen',
                    'log_pl_insol', 'log_pl_eqt',
                    'log_pl_ratdor', 'log_st_teff',
                    'log_st_rad', 'log_st_mass']
new_df_log = df[selected_columns]

new_df_log.head(10)

Show hidden output

new_df_log.isnull().sum()

Show hidden output

new_df_log.describe()

Show hidden output

from sklearn.preprocessing import StandardScaler

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("/content/Imputed_exoplanet_Dataset.csv")

columns_to_transform = ['pl_orbper', 'pl_orbsmax', 'pl_rade', 'pl_masse', 'pl_dens',
                        'pl_orbeccen', 'pl_insol', 'pl_eqt', 'pl_ratdor', 'st_teff',
                        'st_rad', 'st_mass']
for column in columns_to_transform:
    new_column_name = 'log_' + column
    # Replace zeros and negative values with a small positive value before taking the log
    df[new_column_name] = np.log(df[column].replace(0, 1e-10).clip(lower=1e-10))
    #The above line replaces 0 with a very small value and clips the values at lower=1e-10

selected_columns = ['log_pl_orbper', 'log_pl_orbsmax',
                    'log_pl_rade', 'log_pl_masse',
                    'log_pl_dens', 'log_pl_orbeccen',
                    'log_pl_insol', 'log_pl_eqt',
                    'log_pl_ratdor', 'log_st_teff',
                    'log_st_rad', 'log_st_mass']
new_df_log = df[selected_columns]

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(new_df_log)

```

https://colab.research.google.com/drive/1EgW7eY2zmjdxSn0Xw_AYeLkGoXWmAd4#scrollTo=-AqhVUARQW7B&printMode=true

1/2

```

## Import Libraries ##

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

## Import new dataset with planet categories and transformed data ##

df = pd.read_csv("/content/Official Exoplanet Project Data - Transformed_Dataset.csv")

column_names = df.columns
print(column_names)

Show hidden output

df.Planet_Type.value_counts()

Show hidden output

df.Planet_Type.unique()

array(['Terrestrial', 'Gas Giant', 'Neptunian', 'Unknown', 'Super-Earth'],
      dtype=object)

## Remove Planet Types which are "Unknown" ##

df_transformed = df[df.Planet_Type != 'Unknown']

df_transformed = df_transformed.reset_index(drop=True)

df_transformed["Planet_Type"].value_counts()

Show hidden output

len(df_transformed)

2276

## Train and Test the Data ##

X = df_transformed[['Orbit_Semi-Major_Axis', 'Orbital_Period_days', 'Planet_Radius',
                    'Planet_Mass', 'Planet_Density', 'Insolation_Flux',
                    'Ratio_of_Semi-Major_Axis_to_Stellar_Radius',
                    'Stellar_Effective_Temperature ', 'Stellar_Radius', 'Eccentricity']]
y = df_transformed["Planet_Type"]

## Variable Imbalance (SMOTE) ##

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix

import matplotlib.pyplot as plt

count = df_transformed["Planet_Type"].value_counts()

plt.figure(figsize=(6, 4))
plt.pie(count, labels=count.index, autopct='%1.1f%%', startangle=140)
plt.axis('equal')
plt.title('Distribution of Planet Types')
plt.show()

Show hidden output

from imblearn.over_sampling import SMOTE

```

https://colab.research.google.com/drive/1_2hXRhDczPYeYHEWwpYvJZXpFDkW-ffv#scrollTo=EqGcoZSSaCup&printMode=true

1/6

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
```

```
print(y_train_resampled.value_counts())
```

```
Planet_Type
Gas Giant      1364
Neptunian      1364
Terrestrial    1364
Super-Earth    1364
Name: count, dtype: int64
```

```
## 1. SVM OvO ##
```

```
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_model.fit(X_train_resampled, y_train_resampled)
```

```
SVC
```

```
y_pred = svm_model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Show hidden output
```

```
# Confusion Matrix #
```

```
from sklearn import metrics
import seaborn as sns
```

```
unique_classes = y_test.unique()
print(unique_classes)
```

```
['Gas Giant' 'Neptunian' 'Super-Earth' 'Terrestrial']
```

```
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5, 5))
sns.heatmap(cnf_matrix, annot=True, cmap='Reds', fmt='g',
            xticklabels=unique_classes, yticklabels=unique_classes,
            cbar=False)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

```
Show hidden output
```

```
# ROC Curve #
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets, metrics
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets, metrics
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder, label_binarize
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc
from sklearn.multiclass import OneVsRestClassifier
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from itertools import cycle
import pandas as pd
import numpy as np
```

```

le = LabelEncoder()
y_encoded = le.fit_transform(y)

y_test_encoded = le.transform(y_test)
y_binarized = label_binarize(y_test_encoded, classes=np.unique(y_encoded))
n_classes = y_binarized.shape[1]

model = OneVsRestClassifier(SVC(kernel='rbf', probability=True, random_state=42))
model.fit(X_train, y_train)

y_score = model.decision_function(X_test)

fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_binarized[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'darkgreen'])
class_names = le.classes_

plt.figure(figsize=(10, 7))
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f"ROC curve for class {class_names[i]} (AUC = {roc_auc[i]:.2f})")

plt.plot([0, 1], [0, 1], "k--", lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Multi-class ROC Curve - SVM")
plt.legend(loc="lower right")
plt.grid()
plt.show()

```

 [Show hidden output](#)

```

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=le.classes_))

```



	precision	recall	f1-score	support
Gas Giant	0.99	0.95	0.97	320
Neptunian	0.86	0.96	0.91	97
Super-Earth	0.95	0.95	0.95	21
Terrestrial	0.95	1.00	0.97	18
accuracy			0.96	456
macro avg	0.94	0.97	0.95	456
weighted avg	0.96	0.96	0.96	456

2. Decision Tree

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score
import numpy as np
from mlxtend.plotting import plot_decision_regions

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_resampled, y_train_resampled)
y_pred_dt = dt_model.predict(X_test)

dt = DecisionTreeClassifier(criterion='gini', max_depth=2, random_state=30)
dt.fit(X_train, y_train)

```



DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2, random_state=30)

```
y_pred = dt.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print('Accuracy: {:.2f}'.format(accuracy))
```

➦ Accuracy: 0.95

```
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

➦

```
[[320  0  0  0]
 [ 1  96  0  0]
 [ 0  0 20  1]
 [ 0  0  0 18]]
```

	precision	recall	f1-score	support
Gas Giant	1.00	1.00	1.00	320
Neptunian	1.00	0.99	0.99	97
Super-Earth	1.00	0.95	0.98	21
Terrestrial	0.95	1.00	0.97	18
accuracy			1.00	456
macro avg	0.99	0.99	0.99	456
weighted avg	1.00	1.00	1.00	456

```
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from mlxtend.plotting import plot_decision_regions
import numpy as np
```

```
fig, ax = plt.subplots(figsize=(10, 10))
tree.plot_tree(dt, fontsize=10)
plt.show()
```

➦ Show hidden output

```
## Confusion Matrix ##
```

```
y_pred_dt = dt.predict(X_test)
```

```
y_pred_dt = dt.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred_dt)
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu",
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.title("Confusion Matrix - Decision Tree Classifier")
plt.show()
```

➦ Show hidden output

```
y_test_binarized = label_binarize(y_test, classes=dt.classes_)
n_classes = y_test_binarized.shape[1]
```

```
y_score_dt = dt.predict_proba(X_test)
```

```
fpr = dict()
tpr = dict()
roc_auc = dict()
```

```
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_binarized[:, i], y_score_dt[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
```

```
plt.figure(figsize=(10, 7))
```

```
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], lw=2, label=f'ROC curve (area = {roc_auc[i]:.2f}) for class {dt.classes_[i]}')
```

```
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
```